IntroCrypto – Midterm

⚠ Disclaimer

Bei der Erstellung der folgenden Inhalte wurde viel mit LLMs gearbeitet. Keine Gewähr auf Richtigkeit und Vollständigkeit — die VO-Folien sind die Wahrheit. Alle Bilder/Screenshots stammen aus den Folien zu IntroCrypto (TU Wien, 2026W). Bei Fragen / Verbesserungsvorschlägen bitte per Discord (@xmozz) melden.

ⓘ Inhalt:

01 · Intro & historische Chiffren
02 · Moderne Krypto, Perfect Secrecy & OTP
03 · Blockchiffren: DES & AES
04 · Stream Ciphers & Computational Security
05 · Pseudorandomness & Security Proofs
06 · PRFs & CPA-secure Encryption
07 · Modes of Operation & CCA-Security

ⓘ Additional_Files

Zusatzmaterial
SPN – Erklärung Schritt für Schritt
Feistel-Netzwerk – Erklärung Schritt für Schritt

01 · Intro & historische Chiffren

Intro – Geschichte & moderne Krypto

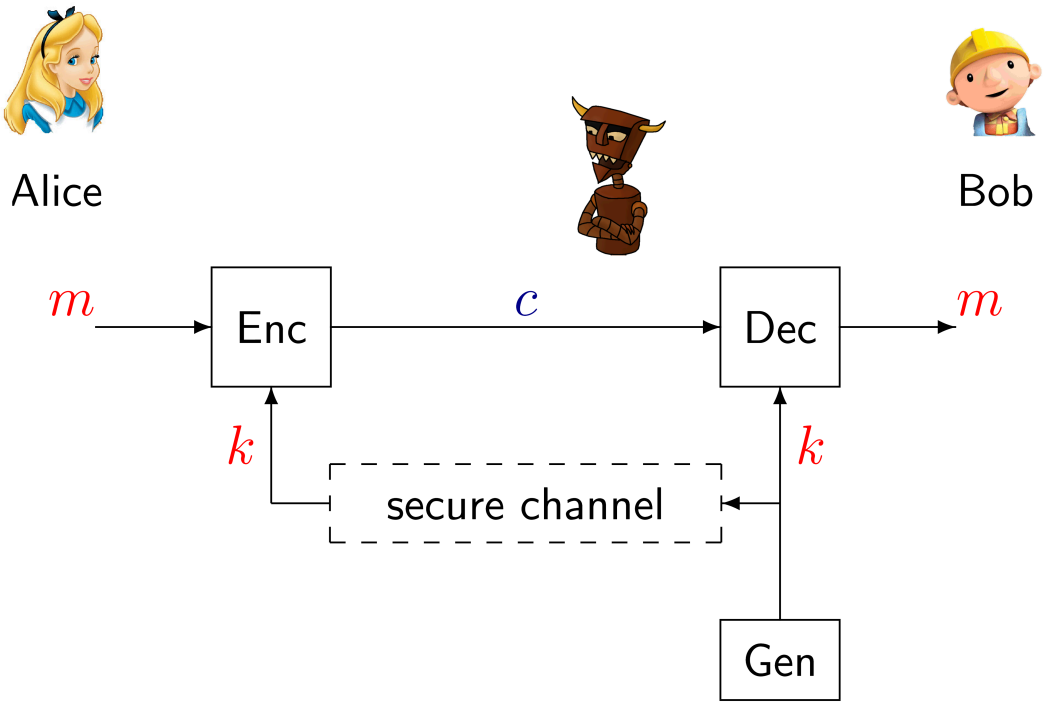
- **History:**
  - Verschlüsselung seit Antike
  - bis 1970er v.a. Militär / Regierungen
  - 20. Jh.: rigoroser Zugang → **Kryptographie ist jetzt Wissenschaft**
  - 1975: **DES** — vollständiger Zeitstrahl bis heute: Landkarte (VO 2)
- **Moderne Krypto:**
  - Ausgangspunkt: **Private-Key Encryption** (geheimer Key zwischen Parteien)
  - daraus: Authentication / Integrity, Public-Key Krypto, E-Cash / Cryptocurrencies
  - fortgeschritten: IBE, MPC, FHE, ZKP, …
- **Referenzbuch:** Katz, Lindell: *Introduction to Modern Cryptography* (3rd edition)
  - VO-Reihe: History, OTP → Blockchiffren (PRFs): DES, AES → Computational Security, Reduktionsbeweise → Private-Key Encryption, MACs → Hash → Zahlentheorie, Public-Key → Signaturen
- **Topics-Matrix:**
  - *Secrecy:* Private-Key Encryption, Public-Key Encryption
  - *Integrity:* MACs, Digitale Signaturen
  - *Bausteine:* Blockchiffren (PRFs), Hashfunktionen, Zahlentheorie
- **Ziele:**
  - Krypto *verstehen* wie sie real verwendet wird
  - Krypto *benutzen*: welche Primitive gibt es, welche präzisen Garantien geben sie (genutzte Schemes + **provable security** Paradigma)

Private-Key Encryption – Modell & Definition

- **Modell (symmetrisch):** Alice + Bob teilen Key k über sicheren Kanal, Eve sieht nur c
  - $Gen \to k$, Alice: $c \leftarrow Enc_k(m)$, Bob: $m := Dec_k(c)$
- symmetrisch = gleicher Key zum Ver-/Entschlüsseln

Private-key encryption

("Symmetric encryption")



17

Private-Key Scheme – Def. §1.2

- Scheme = Nachrichtenraum M + 3 Algorithmen:
- Gen (probabilistisch): gibt Key k aus ($k \leftarrow Gen$)
- Enc: Eingabe k, m, Ausgabe c ($c \leftarrow Enc_k(m)$)
- Dec: Eingabe k, c, Ausgabe m ($m := Dec_k(c)$)
- **Korrektheit:** für alle $k \leftarrow Gen$, alle $m \in M$: $Dec_k(Enc_k(m)) = m$

Kerckhoffs' Prinzip (1883)

Kerckhoffs

Enc **und** Dec **sind öffentlich** – Sicherheit von m gegeben c hängt **allein von Geheimhaltung von** k ab. System darf nicht auf Geheimhaltung angewiesen sein, auch wenn es in Feindeshand fällt.

- **Argumente:**
  - Keys geheim halten leichter als Algorithmen (Reverse Engineering – Algorithmus bleibt nicht geheim)
  - Key tauschen leichter als Scheme tauschen

- Schemes standardisierbar → Kompatibilität + öffentliche Prüfung (public scrutiny)

Historische Chiffren – Permutation (Scytale)

- **Scytale (Spartaner):** Key = *Umfang* des Stabs
  - Nachricht zeilenweise um Stab schreiben, abgewickelt spaltenweise lesen
  - Beispiel $k = 3$: `atymbhhaeudlrlp`, $k = 4$: `ardemtllhbypuah` (gleicher Text → anderer Cipher je nach Key)
  - nur *bestimmte* Permutationen möglich (kleiner Keyspace)
- **Prinzip Permutation:** Key definiert Umordnung (*rearranging*) der Nachricht

Ancient ciphers

- Scytale (Spartans)
  key = "circumference"





```
    |   |   |   |   |   |
    | a | m | h | u |   |
|---| r | t | b | a |---|
|   | d | l | y | h |
|   | e | l | p |   |
|   |   |   |   |   |
```

$k = 4$

$\rightarrow$ `ardemtllhbypuah`

(only certain permutations possible)

21 - 2

Monoalphabetische Substitution & Shift

- **Monoalphabetisch** (z.B. Caesar, ROT13):
  - Alphabet $\Sigma = \{A, \ldots, Z\}$, Key = Bijektion $\pi : \Sigma \to \Sigma$
  - $Enc(\pi, m_1 \ldots m_n) = \pi(m_1) \ldots \pi(m_n)$
  - $Dec(\pi, c_1 \ldots c_n) = \pi^{-1}(c_1) \ldots \pi^{-1}(c_n)$
  - Beispiel-π: $A \to K, B \to X, C \to M, D \to Q, \ldots$
- **Shift Cipher** (Teilmenge der Substitution):
  - $A \leftrightarrow 0, \ldots, Z \leftrightarrow 25$, Key $k \in \{0, \ldots, 25\}$, Caesar: $k = 3$
  - $Enc(k, m) := (m + k) \bmod 26$
  - $Dec(k, c) := (c - k) \bmod 26$

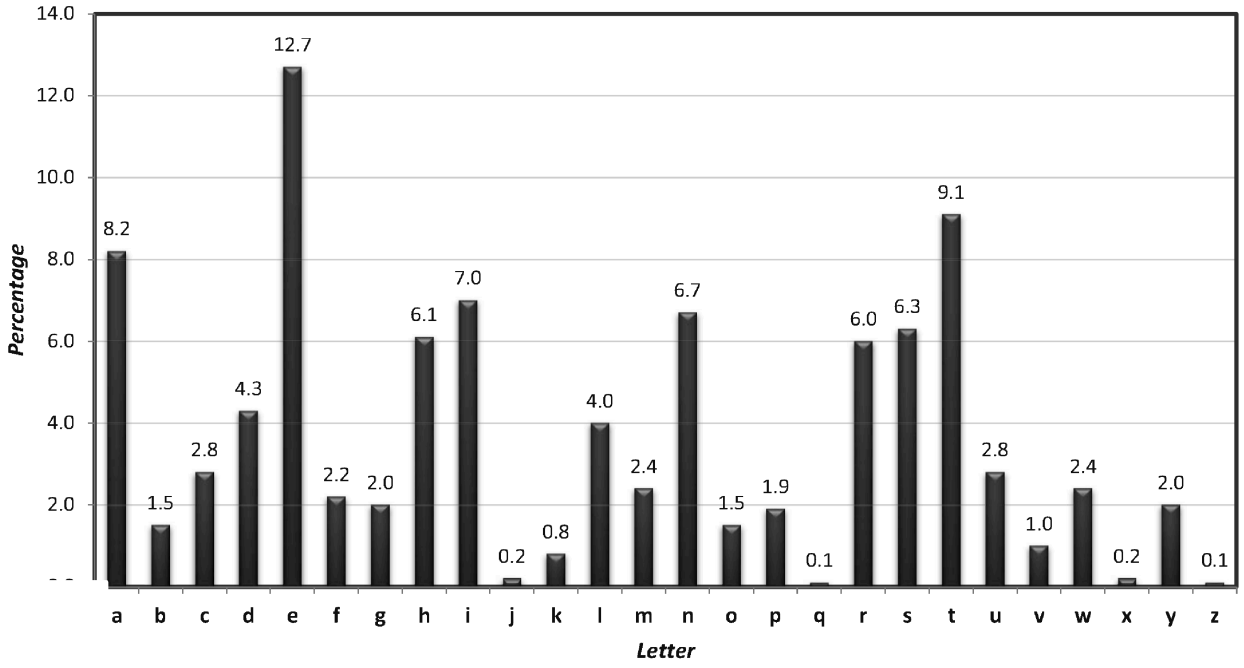
Kryptoanalyse I – Brute Force & Keyspace

- **Shift brechen:**
  - wie viele Keys? nur **26**
  - gegeben c: alle k probieren, welches $m = Dec(k, c)$ ergibt Sinn?
  - heißt **Brute-Force / Exhaustive Search**
- **Lehre:** Keyspace muss groß genug sein
- **Einordnung:**
  - Mifare Crypto-1: 48 Bit (2^{48}, auf Handy brechbar)
  - DES: 56 Bit (2^{56}, auf Laptop brechbar)
  - AES: 2^{128} oder 2^{256}

> **Faustregel Keyspace**
>
> 26 Keys = sofort tot. 48/56 Bit = heute zu klein. 128+ Bit = Standard.

Kryptoanalyse II – Frequenzanalyse

- **Allgemeine Substitution brechen** (nicht nur Shifts):
  - jeder Buchstabe → anderer (beliebige Bijektion)
  - Keys: $26! > 2^{88}$ (Anzahl Permutationen) → Brute Force unmöglich
  - trotzdem gebrochen via **statistische Analyse** (Häufigkeit der Buchstaben)
  - (Idee: E ist am häufigsten etc. – Struktur der Sprache bleibt erhalten)



Polyalphabetisch & Vigenère (1553)

- **Polyalphabetisch:** Key $k = (\pi_1, \ldots, \pi_\ell)$

  - $Enc(k, m_1 \dots m_n) = \pi_1(m_1)\pi_2(m_2) \dots \pi_\ell(m_\ell)\pi_1(m_{\ell+1}) \dots$ (Substitution hängt von Position ab)
- **Vigenère** (Teilmenge davon):
  - Beispiel MESSAGE CLAIR + Key CLE…:
  - $MESSAGECLAIR$ $(12, 04, 18, 18, 00, 06, 04, 02, 11, 00, 08, 17)$ + $CLECLECLECLE$ ($02, 11, 04, 02, 11, 04, 02, 11, 04, 02, 11, 04$) = $OPWULKGNPCTV$ ($14, 15, 22, 20, 11, 10, 06, 13, 15, 02, 19, 21$) mit $A \leftrightarrow 0, \dots, Z \leftrightarrow 25$
  - Key *wiederholt* sich (short Key, long Message)
- **Kryptoanalyse:** auch statistische Angriffe (z.B. Keylänge schätzen, dann pro Position Frequenzanalyse)

M	E	S	S	A	G	E	C	L	A	I	R
C	L	E	C	L	E	C	L	E	C	L	E
O	P	W	U	L	K	G	N	P	C	T	V
12	04	18	18	00	06	04	02	11	00	08	17
02	11	04	02	11	04	02	11	04	02	11	04
14	15	22	20	11	10	06	13	15	02	19	21

A ↔ 0
⋮
Z ↔ 25

Vernam Cipher → One-Time Pad Ausblick

- **Vernam** (vgl. OTP):
  - Key = *zufällige* Folge **gleicher Länge** wie Message
  - $c_i := (m_i + k_i) \bmod 26$
  - Beispiel Folie: gleiches $PLAINTEXT$ mit anderem Key → völlig anderes $ENCRYPTED$ vs. $DIFFERENT$ ($PMALTPXXU$ vs. $BFYMUYPRK$ etc.)
  - (Intuition: langer Zufalls-Key → keine Wiederholung wie bei Vigenère → Frequenzanalyse greift nicht)

Überblick – Merkschema

- **Permutation** (Blöcke/Buchstaben umordnen), z.B. Scytale (wenige Permutationen)
- **Substitution** (Blöcke/Buchstaben durch andere ersetzen):
  - *monoalphabetisch:* Shift (Caesar) oder zufällige Bijektion
  - *polyalphabetisch* (hängt von Stelle ab): Vigenère (Key wiederholt sich) oder Vernam (langer Key)
- nächste VO: Vernam → **One-Time Pad**, moderne Krypto → VO 2: Das One-Time Pad

✓ Kern für die Prüfung

- Scheme-Def. + Korrektheit: $Dec_k(Enc_k(m)) = m$ hinschreiben können
- Kerckhoffs + 3 Argumente erklären
- 26 vs. $26! > 2^{88}$ vs. Frequenzanalyse unterscheiden
- Vigenère-Formel + Key-Wiederholung, Vernam-Formel $c_i := (m_i + k_i) \bmod 26$ + gleichlanger Zufalls-Key
- Keygrößen: 2^{48} / 2^{56} / 2^{128} einordnen

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025)

- `1a` „nur `Gen` ist randomisiert, `Enc` / `Dec` müssen deterministisch sein“ → **F** (`Enc` **darf** randomisiert sein; nur `Dec` muss deterministisch sein)
- `1b` „polyalphabetisch (Vigenère) ⇒ immun gegen Frequenzanalyse“ → **F** (der *kurze, wiederholte* Key macht angreifbar — nicht die Klasse)

Sample 2026W

- `1a` Caesar-Chiffre über dem 1-Buchstaben-Raum $\{A, \ldots, Z\}$ perfectly secret? → **T** (26 gleichwahrscheinliche Keys + eindeutiges k pro (m, c) ⇒ Shannon 2.12 erfüllt; „perfectly secret“ scheitert erst an *langen* Nachrichten)

02 · Moderne Krypto, Perfect Secrecy & OTP

Landkarte: moderne Krypto in der Zeit

Jahr	Meilenstein	Wer
1917	**Vernam Cipher** (Vorläufer des OTP)	Vernam
1949	Informationstheorie, **perfect secrecy**	Shannon
1976	**Public-Key**-Kryptographie	Diffie, Hellman
1977	**DES** (Feistel-Netzwerk) als Encryption-Standard	Feistel
1978	**RSA**	Rivest, Shamir, Adleman
1982	**E-Cash**	Chaum
1984	**Semantic Security**	Goldwasser, Micali
1986	Digitale **Signaturen**	Goldwasser, Micali, Rivest
1986	**MPC** (multiparty computation)	Yao, Goldreich, Micali, Wigderson
1989	**Zero-Knowledge**-Proofs	Goldwasser, Micali, Rackoff
1996	Lattice-based crypto	Ajtai, Dwork
2000 / 01 / 15	**AES** / SHA-2 / SHA-3	Standards
2008	**Bitcoin**	Nakamoto
2009	**FHE** (fully homomorphic encryption)	Gentry
2013	**Obfuscation**	Garg, Gentry, Halevi, Raykova, Sahai, Waters
2016	**CKKS** → FHE wird praktisch	
2020	Obfuscation aus **Standardannahmen**	Jain, Lin, Sahai
heute	zk-SNARKs / STARKs praktisch; **ML-KEM, ML-DSA, FN-DSA** standardisiert (post-quantum)	

Der Paradigmenwechsel: von Kunst zu Wissenschaft

- Bis ~1980: Krypto ist **Handwerk** — Scheme bauen, Scheme brechen, nächstes Scheme. "Sicher" ist ein Gefühl, kein Prädikat.
- Ab ~1980: **art → science**. Sicherheit wird zu einer *beweisbaren* Aussage.
- Name des Programms: **Provable Security** (= *reductionist security*), Buchkapitel **§1.4**.

Die drei Säulen

Säule	Beantwortet die Frage	Beispiel aus der VO
Definitions	Was ist das **Sicherheitsziel**, was das **Threat Model**?	"Unterscheide zwei Verschlüsselungen" (*indistinguishability*) unter *chosen-plaintext attack*
Assumptions	Welches Problem ist **effizient unlösbar**?	Factoring großer Zahlen ist schwer
Security proof	Warum erfüllt meine Konstruktion die Definition?	mathematischer Beweis / Reduktion

> **Der Dreiklang in einem Satz**
>
> "Scheme X erfüllt Definition Y, **falls** Annahme Z gilt."
> Ein Sicherheitsbeweis behauptet **nie** mehr als das — die Annahme ist immer mitzitiert.

Wozu Definitionen? (Folie 6)

- **verstehen**, was man überhaupt erreichen will
- Schemes **evaluieren und vergleichen** (statt Bauchgefühl)
- Schemes als **Baustein** verwenden → **Modularität**: daraus entsteht der ganze Krypto-Baukasten (Blockchiffre → PRF → Encryption → MAC → …)

Annahmen (Folien 7-1 / 7-2)

- Eine Annahme ist die Aussage: **dieses Problem ist nicht effizient lösbar** (z.B. Factoring).
- Präzise definiert ⇒ **studierbar, validierbar, vergleichbar** (man kann Schemes nach der *Stärke* ihrer Annahme ordnen).
- **P vs NP:** Gibt es Probleme, bei denen *Lösung finden* schwerer ist als *Lösung prüfen*?
- **Zentrale Konsequenz:** Die Sicherheit **jedes interessanten** Schemes impliziert $P \neq NP$.
  ⇒ Da $P \neq NP$ unbewiesen ist, braucht man **hardness assumptions** — Sicherheit lässt sich nie "absolut" beweisen.
- Woher die Annahmen kommen:
  - **Public-Key:** "mathematisch" (Factoring, diskreter Logarithmus, …) — gut verstanden
  - **symmetrisch:** typischerweise **ad-hoc** (z.B. "AES verhält sich wie eine zufällige Permutation")

Sicherheitsbeweise — und ihre Grenzen (Folie 8)

Ein *beweisbar sicheres* Scheme kann in der Praxis trotzdem scheitern, wenn:

- die **Annahme falsch** war — z.B. Quantencomputer faktorisieren effizient ⇒ RSA bricht

- die **Definition die Realität nicht abbildet** — z.B. Scheme ist bewiesen sicher, *solange der Key nur einmal benutzt wird*, in der Praxis sieht der Angreifer aber zwei Ciphertexte

Vorteile (warum man es trotzdem macht):

- wohldefinierte, isolierte **Analyseprobleme** (statt "ich glaube, es ist sicher")
- Sicherheit eines **neuen** Schemes auf **gut studierte** Annahmen zurückführen

> ⚠ **Wichtig**
>
> "beweisbar sicher" ≠ "unknackbar". Es heißt: *unter Annahme X erfüllt es Definition Y*. Wer den Zusatz weglässt, hat die Aussage zerstört.

Security Definitions: das Angreifermodell

Eine Definition legt **drei** Dinge fest.

Achse	Ausprägungen
Wissen	**Kerckhoffs' Prinzip**: der Angreifer **kennt das Scheme** — nur der Key ist geheim (Details + Begründung: VO 1, Kerckhoffs)
Ziel	Key finden · ganzen Plaintext rekonstruieren · **einen einzigen Buchstaben** raten · **irgendeine Information** über den Plaintext erhalten
Macht	einen Ciphertext sehen · **viele** Ciphertexte · Plaintext/Ciphertext-Paare kennen · Plaintexte **wählen** (chosen-plaintext) · zusätzlich ein **Decryption Oracle** fragen (→ CPA / CCA später)

> 🔥 **Warum ist "any information" das richtige Ziel?**
>
> Weil es das **stärkste** Ziel ist: kann der Angreifer nicht einmal *ein Bit* Information gewinnen, dann kann er erst recht nicht den Key oder den ganzen Plaintext finden. Alle schwächeren Ziele sind damit abgedeckt — deshalb definiert man Sicherheit immer gegen das stärkste Ziel.

Schlechte Schemes als Anschauung (Folie 10):

- $Enc(k, m) := m$ — offensichtlich: der Key wird ignoriert
- $Enc(k, m_1 \parallel m_2) := AES(k, m_1) \parallel m_2$ — der zweite Block geht **im Klartext** raus; man sieht Struktur/Wiederholungen. (Genau dieses Problem löst später ein *mode of operation*.)

Werkzeugkasten: Wahrscheinlichkeit (§A.3)

> ⓘ **Warum dieser Abschnitt?**
>
> Perfect secrecy ist **keine neue Idee, sondern reine Wahrscheinlichkeitsrechnung**

Die fünf Formeln

(1) $Pr[X = x]$ — Wahrscheinlichkeit, dass die Zufallsvariable X den Wert x annimmt.
Wofür: Jede Sicherheitsaussage ist eine Aussage über **Verteilungen** (M, K, C), nie über einen einzelnen Wert. Erst damit wird „der Angreifer weiß etwas“ überhaupt formalisierbar.

(2) Bedingte Wahrscheinlichkeit: $Pr[X = x \mid Y = y] := \dfrac{Pr[X = x \wedge Y = y]}{Pr[Y = y]}$ — Wahrscheinlichkeit von X, **wenn man Y schon sieht**.
Wofür: Genau die Lage des Angreifers: er sieht c und fragt nach m ⇒ $Pr[M = m \mid C = c]$. Diese Größe steht wörtlich in **Def. 2.3**.

(3) Unabhängigkeit: X, Y unabhängig

$$\iff \forall x, y : \; Pr[X = x \mid Y = y] = Pr[X = x]$$

Y verrät nichts über X ⇒ die Bedingung darf **wegfallen**.
Wofür: Modellannahme $K \perp M$ (der Key hängt nicht von der Nachricht ab). Im OTP-Beweis ist das der Schritt $Pr[K = m \oplus c \mid M = m] = Pr[K = m \oplus c]$ — ohne ihn kommt man nicht weiter.

(4) Totale Wahrscheinlichkeit:

$$\sum_{y_i} Pr[Y = y_i] = 1$$

und

$$Pr[X = x] = \sum_{y_i} Pr[X = x \mid Y = y_i] \cdot Pr[Y = y_i]$$

„alle Fälle durchgehen“: aus bedingten Wahrscheinlichkeiten die **unbedingte** ausrechnen.
Wofür: Beweisschritt:

$$Pr[C = c] = \sum_{m'} Pr[C = c \mid M = m'] \cdot Pr[M = m']$$

Auch im Shift-Beispiel:

$$Pr[C = F] = Pr[C = F \mid M = B] \cdot Pr[M = B] + Pr[C = F \mid M = D] \cdot Pr[M = D]$$

(5) Bayes:

$$Pr[X = x \mid Y = y] = Pr[Y = y \mid X = x] \cdot \frac{Pr[X = x]}{Pr[Y = y]}$$

die **Richtung der Bedingung umdrehen**.
Wofür: Das ist der Trick des ganzen Beweises: $Pr[C = c \mid M = m]$ (Verschlüsselung) können wir ausrechnen, gebraucht wird aber $Pr[M = m \mid C = c]$ (Angreifer). Bayes ist die Brücke — Schritt **(4)** in Thm. 2.10.

Kochrezept "perfect secrecy beweisen"

1. $Pr[C = c \mid M = m]$ hinschreiben und mit **Key-Uniformität** (1) + **Unabhängigkeit** (3) zu $\frac{1}{2^\ell}$ machen.
2. $Pr[C = c]$ über **totale Wahrscheinlichkeit** (4) ausrechnen — es kommt ebenfalls $\frac{1}{2^\ell}$ raus.
3. **Bayes** (5) anwenden; die $\frac{1}{2^\ell}$ kürzt sich weg.
4. Ergebnis: $Pr[M = m \mid C = c] = Pr[M = m]$ ⇒ Definition erfüllt.

Bayes einmal konkret (Shift-Chiffre, $Pr[M = B] = 0.7$)

Angreifer sieht $c = F$. Was weiß er jetzt über M?

$$Pr[M = B \mid C = F] = \underbrace{Pr[C = F \mid M = B]}_{= Pr[K=4] \,=\, 1/26} \cdot \frac{Pr[M = B]}{Pr[C = F]} = \frac{1}{26} \cdot \frac{0.7}{1/26} = 0.7$$

Also **exakt das Vorwissen** — der Ciphertext hat nichts geändert. Genau das *ist* perfect secrecy, hier einmal ausgerechnet statt nur behauptet.

Perfect Secrecy — die Idee

Intuition

Egal was der Angreifer schon weiß — ein Ciphertext soll **keine zusätzliche Information** über den darunterliegenden Plaintext leaken.

Formalisierung über drei Zufallsvariablen:

- M = **Vorwissen** über die Nachricht (Verteilung!), z.B. $Pr[M = \text{"buy"}] = 0.7$, $Pr[M = \text{"sell"}] = 0.3$
- K = Key-Wert, $Pr[K = k] = Pr[k \leftarrow Gen]$
- Annahme: K **und** M **sind unabhängig** (der Key wird nicht passend zur Nachricht gewählt)
- C = resultierender Ciphertext: $m \sim M$, $k \leftarrow Gen$, $c \leftarrow Enc_k(m)$

Beispiel 2.1 — Shift Cipher ($Pr[K = k] = 1/26$)

Mit $Pr[M = B] = 0.7$, $Pr[M = D] = 0.3$: welches c entsteht bei $C = F$?

- F entsteht aus B mit Key 4 ($B + 4 = F$) **oder** aus D mit Key 2 ($D + 2 = F$)
- also $Pr[C = F] = Pr[M = B \wedge K = 4] + Pr[M = D \wedge K = 2]$
- M, K unabhängig $\Rightarrow = 0.7 \cdot \frac{1}{26} + 0.3 \cdot \frac{1}{26} = (0.7 + 0.3) \cdot \frac{1}{26} = \frac{1}{26}$

Und jetzt der Punkt: mit $Pr[M = B] = 0.1$, $Pr[M = D] = 0.9$ und $C = G$ kommt genauso $\frac{1}{26}$ raus.

> **Was das Beispiel zeigt**
>
> Die **Ciphertext-Verteilung ist uniform (hier $\frac{1}{26}$), völlig unabhängig von der Plaintext-Verteilung**. Der Angreifer sieht c und kann daraus nichts über M ableiten — genau das ist perfect secrecy.

> **Perfect Secrecy — Def. 2.3**
>
> Ein Encryption Scheme ist **perfectly secret**, wenn für **jede** Wahrscheinlichkeitsverteilung über M, **jedes** $m \in M$ und **jedes** c mit $Pr[C = c] > 0$ gilt:
>
> $$Pr[M = m \mid C = c] = Pr[M = m]$$
>
> *(Der Angreifer sieht dabei nur **einen** Ciphertext — das ist die Machtstufe dieser Definition.)*

> **Äquivalente Form, die man zum Rechnen nimmt**
>
> Per Bayes ist die Bedingung äquivalent zu
>
> $$Pr[C = c \mid M = m] = Pr[C = c]$$
>
> "Der Ciphertext sieht für jede Nachricht gleich aus." Im OTP-Beweis rechnet man genau diese Richtung.

Das One-Time Pad

- **"Red phone"** zwischen Kreml und Weißem Haus — das historische Einsatzszenario
- **perfectly secure** (wenn korrekt benutzt) · aber **not practical**
- Name: *Pad* = Notizblock mit Zufallszahlen, *one-time* = jede Seite nur **einmal**

XOR als Rechenoperation

$$x \oplus y = (x + y) \bmod 2$$

$\oplus$	0	1
0	0	1
1	1	0

Eigenschaften, die man ständig braucht: kommutativ, assoziativ, **selbstinvers** ($x \oplus x = 0$), $x \oplus 0 = x$.

Konstruktion

- Message space: $M := \{0,1\}^\ell$
- $Gen(\ell)$: $k \leftarrow \{0,1\}^\ell$ — **uniform zufällig**, gleiche Länge wie die Nachricht
- $Enc(k, m) := k \oplus m = (k_1 \oplus m_1) \parallel \cdots \parallel (k_\ell \oplus m_\ell)$
- $Dec(k, c) := k \oplus c$

Korrektheit (immer hinschreiben können):

$$Dec(k, Enc(k, m)) = k \oplus (k \oplus m) = (\underbrace{k \oplus k}_{=0\ldots0}) \oplus m = m$$

Warum es überhaupt funktioniert

Zu **jedem** Paar (m, c) gibt es **genau einen** passenden Key: $k = m \oplus c$. Weil k uniform gewählt ist, ist auch c uniform — und zwar **unabhängig von** m. Der Ciphertext "verrät" also nichts, weil für jeden möglichen Plaintext genau ein gleich wahrscheinlicher Key existiert.

Beweis: Das OTP ist perfectly secret (Thm. 2.10)

Theorem 2.10

Das One-Time Pad ist perfectly secret. Mit $Gen(\ell) : k \leftarrow \{0,1\}^\ell$, $Enc_k(m) := k \oplus m$, $Dec_k(c) := k \oplus c$.

Beweisstrategie in einem Satz: Zeige $Pr[C = c \mid M = m] = Pr[C = c]$; dann kürzt sich in Bayes alles weg. *(Die benutzten Regeln sind im Werkzeugkasten oben unter (1)–(5) nummeriert.)*

Sei M, m und c beliebig, aber **fest** gewählt.

(*) Keys sind uniform: $Pr[K = k] = \frac{1}{2^\ell}$ für alle k.

(**) Ciphertext gegeben der Plaintext:

$$Pr[C = c \mid M = m] = Pr[K \oplus M = c \mid M = m] = Pr[K \oplus m = c \mid M = m]$$

$$= Pr[K = m \oplus c \mid M = m] \overset{K,M \text{ unabh.}}{=} Pr[K = m \oplus c] \overset{(*)}{=} \frac{1}{2^\ell}$$

(***) Ciphertext marginal (totale Wahrscheinlichkeit):

$$Pr[C = c] = \sum_{m'} Pr[C = c \mid M = m'] \cdot Pr[M = m'] \overset{(**)}{=} \frac{1}{2^\ell} \cdot \sum_{m'} Pr[M = m'] = \frac{1}{2^\ell}$$

(4) Bayes:

$$Pr[M = m \mid C = c] = \frac{Pr[C = c \mid M = m] \cdot Pr[M = m]}{Pr[C = c]}$$

$$\overset{(**),(***)}{=} \frac{\frac{1}{2^\ell} \cdot Pr[M = m]}{\frac{1}{2^\ell}} = Pr[M = m]$$

Da M, m, c beliebig waren, gilt die Gleichung für **alle** Verteilungen über M ⇒ perfectly secret. ■

Key-Reuse: das OTP zerstört sich selbst

Benutzt man den Key **zweimal**, fällt er aus der Rechnung:

$$c = k \oplus m, \quad c' = k \oplus m' \;\Rightarrow\; c \oplus c' = \underbrace{k \oplus k}_{=0\ldots0} \oplus m \oplus m' = m \oplus m'$$

Der Angreifer kennt also $m \oplus m'$ — **ohne** den Key. Bei bekanntem Nachrichtenformat genügt das:

- m: "B" $= 100\,0010$ oder "S" $= 101\,0011$ ("buy" / "sell")
- m': "Y" $= 101\,0110$ oder "N" $= 100\,1110$ ("yes" / "no")
- $c \oplus c' = 001\,0100 \Rightarrow$ **eindeutig** $m = B, m' = Y$
- $c \oplus c' = 000\,0101 \Rightarrow$ passt nicht auf eine der Kombinationen → man muss weiter raten (aber man hat schon massiv Information gewonnen)

> **Merke**
>
> Zwei OTP-Ciphertexte mit gleichem Key verhalten sich wie eine **Vigenère-Chiffre** — das Muster des Plaintexts (XOR der Nachrichten) bleibt erhalten. Genau davor hat die VO 1 (Vigenère) gewarnt.

Nachteile des OTP (die eigentliche Pointe)

Nachteil	Konsequenz
Key ist **so lang wie die Nachricht**	kein kurzer Schlüssel, kein Schlüsselaustausch "mal eben"
Key nur **einmal** benutzbar	Key-Verbrauch = Datenmenge
braucht **lange echte Zufallswerte**	echte Zufälligkeit ist teuer/aufwendig
Key-Distribution	wie kommt der Pad sicher zum Empfänger?
Key-Destruction	benutzte Pads müssen vernichtet werden

Grenzen von perfect secrecy (Shannon)

Theorem 2.11

Für **jedes** perfectly secret Encryption Scheme gilt: $|K| \geq |M|$.

Beweisidee: Sei c ein Ciphertext mit $Pr[C = c] > 0$. Damit $Pr[M = m \mid C = c] = Pr[M = m] > 0$ gelten kann, muss es zu **jedem** m einen Key k mit $Enc_k(m) = c$ geben (sonst wäre m bei diesem c unmöglich). Für $m_1 \neq m_2$ müssen diese Keys **verschieden** sein (ein k kann m_1 nicht auf zwei verschiedene c abbilden). Also injektive Abbildung $M \to K$ ⇒ $|K| \geq |M|$.

Theorem 2.12 (Shannon)

Für ein Scheme mit $|M| = |K| = |C|$ gilt: **perfectly secret** $\iff$

1. jeder Key ist **gleichwahrscheinlich**, und
2. für alle m und c gibt es **genau ein** k mit $Enc_k(m) = c$.

Die Konsequenz — und der Übergang zur nächsten VO

Perfect secrecy ist **nur gegen Angreifer mit unendlicher Rechenkraft** formuliert (keine Effizienz im Spiel). Und sie erzwingt einen Key, der so lang wie die Nachricht ist und nur einmal verwendet wird.
⇒ Für **praktische** Schemes braucht man Sicherheit gegen **beschränkte** Angreifer: **Computational Security** → VO 4. Das ist der ganze Grund, warum es Block- und Streamchiffren gibt.

Kompakt-Tabelle zum Wiederholen

Begriff	Kernaussage	Formel
Provable Security	Definition + Annahme + Beweis	"X erfüllt Y, falls Z"
Kerckhoffs	Angreifer kennt das Scheme, nur der Key ist geheim	—
Perfect Secrecy	Ciphertext leakt keine Zusatzinfo	$Pr[M = m \mid C = c] = Pr[M = m]$
OTP Encryption	XOR mit uniformem Key gleicher Länge	$c = k \oplus m$
OTP Korrektheit	XOR ist selbstinvers	$k \oplus (k \oplus m) = m$
OTP-Sicherheit	Keys uniform ⇒ c uniform, unabhängig von m	$Pr[C = c] = \frac{1}{2^\ell}$
Key-Reuse	Key kürzt sich raus	$c \oplus c' = m \oplus m'$
Shannon 2.11	Key mindestens so lang wie Message	$\|K\| \geq \|M\|$
Shannon 2.12	Gleichheit $\iff$ uniforme Keys + eindeutiges k pro (m, c)	—
Bayes	Richtung der Bedingung umdrehen	$Pr[X = x \mid Y = y] = Pr[Y = y \mid X = x] \cdot \frac{Pr[X=x]}{Pr[Y=y]}$
Totale W.	unbedingte aus bedingten W.	$Pr[X = x] = \sum_{y_i} Pr[X = x \mid Y = y_i] \cdot Pr[Y = y_i]$

✓ Kern für die Prüfung

- **Dreiklang** Definition / Annahme / Beweis + je ein Beispiel nennen können
- $P \neq NP$**-Argument**: ohne hardness assumption keine beweisbare Sicherheit
- **Angreifermodell** in drei Achsen (Wissen / Ziel / Macht) + Kerckhoffs
- **Def. 2.3** exakt hinschreiben ($\forall$ Verteilung, $\forall m$, $\forall c$ mit $Pr[C = c] > 0$) und die äquivalente Form $Pr[C = c \mid M = m] = Pr[C = c]$ kennen
- **OTP**: $c = k \oplus m$, Korrektheit, und **Thm. 2.10 komplett** mit $(*)$, $(**)$, $(***)$ und Bayes
- **Wahrscheinlichkeitsregeln** (bedingte W., Unabhängigkeit, totale W., Bayes) parat haben *und* im Beweis benennen können, welcher Schritt welche Regel ist
- **Key-Reuse**: $c \oplus c' = m \oplus m'$ + konkretes Bit-Beispiel
- **Shannon 2.11/2.12**: $|K| \geq |M|$, Gleichheitsfall, und die Begründung, warum das Computational Security motiviert

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025)

- `1c` „perfectly secret ⟺ perfectly indistinguishable“ → **T** (Lemma 2.7)
- `1d` „Shannon: $|M| = |K| = |C|$ + eindeutiges k pro (m, c) ⇒ perfectly secret“ → **F** (die Bedingung „**jeder Key gleichwahrscheinlich**“ fehlt — genau der Punkt, den Thm. 2.12 zusätzlich verlangt)

Sample 2026W

- `3)` **große Frage (4 P.)** perfectly secret: Π mit $k \leftarrow \{0,1\}^\ell$, $Enc_k(m) = k_{|m|} \oplus m$ für $m \in \{0,1\}^{\leq \ell}$
  - a) *nicht* perfectly secret: $|M| = 2^{\ell+1} - 2 > 2^\ell = |K|$ ⇒ Shannon 2.11 (alternativ direkt: c für $m = 0$ und $m = 1$ ist dasselbe k_1-Bit)
  - b) mit $\mathrm{Pad}(m) = m \| 1 \| 0^{\ell - |m|}$ **ist** Π' perfectly secret (feste Länge $\ell + 1$, uniformer Key $\{0,1\}^{\ell+1}$ ⇒ klassisches OTP auf dem gepaddeten Raum)
  - ⇒ Muster: **Padding auf fixe Länge rettet perfect secrecy**, ungleiche Längen nicht (vgl. `1e` im Sample → VO 4)

03 · Blockchiffren: DES & AES

Blockchiffren – Grundidee (§7.2)

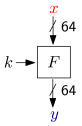
- **Baustein** für Private-Key-Encryption, MACs, …
- verschlüsselt **Blöcke fixer Länge** (bei einer Substitutionschiffre ist der Block **1 Buchstabe**)
- **Generische Angriffe** (unabhängig vom konkreten Design):
  - *Exhaustive key search* ⇒ Schlüsselraum groß genug: $|K| \geq 2^{128}$
    (bei 10^{18} Keys/s = 1 Trillion = 1 quintillion Keys/s dauert es ein **Weltalter**)
  - *Dictionary attack* ⇒ Blockgröße ℓ groß genug: $\ell \geq 64$ **Bit**
    (sonst: Tabelle über alle 2^ℓ möglichen Klartexte/Chiffrate)
- **Setting:** Angreifer kennt den Algorithmus (**Kerckhoffs**), hat Klartext/Chiffrat-Paare **gesehen bzw. gewählt**
- ⇒ die Sicherheit darf **nicht** auf Geheimhaltung des Designs beruhen

Sicherheitsziel: die „ideale Chiffre“

- Idealvorstellung: der **Schlüssel definiert eine zufällige Permutation**

$$F_k : \{0,1\}^\ell \to \{0,1\}^\ell$$

- Es gibt $2^\ell!$ Permutationen; für $\ell = 64$ bräuchte man $|k| \approx 2^{70}$ Bit Schlüssel (≈ **150 Millionen TB**) — völlig unpraktisch
- **Vergleich mit dem One-Time-Pad:** dort bestimmt **ein** x/y-Paar **alle** Paare (denn $k = m \oplus c$ legt den ganzen Pad fest). Bei einer Permutation legt ein Paar nur **einen** Eintrag der Tabelle fest
- **Ziel:** eine effizient berechenbare, *keyed* Permutation F mit **kurzem** Schlüssel konstruieren, die sich **„wie die ideale Chiffre verhält“**



Was heißt „verhält sich wie ideal“?

Für einen effizienten Angreifer, der F_k als Black Box mit $x_i \mapsto y_i$ abfragt, darf **nicht erkennbar** sein, ob er eine echte Permutation mit kurzem Schlüssel vor sich hat oder eine zufällige Permutation. Das ist genau die **PRP**-Annahme (pseudorandom permutation) — formal definiert erst in VO 6, PRP.

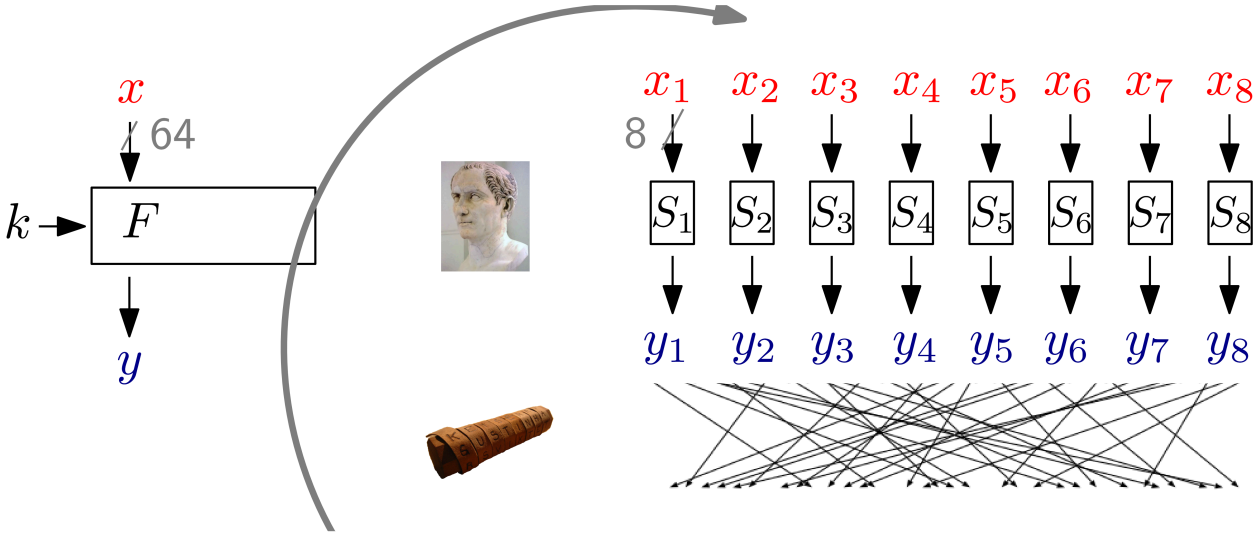
Methoden: Confusion & Diffusion (Shannon)

- **Confusion:** die Beziehung zwischen x und y wird „verschleiert“ ⇒ **Substitution**
- **Diffusion:** jedes Bit von y hängt von **vielen** Bits von x ab ⇒ **Permutation** (Bit-Mixing)

Ziel	Baustein	Wirkung
Confusion	Substitution (S-Box)	Nicht-Linearität
Diffusion	Permutation / lineare Mischung	Ein Eingabebit „verwässert“ über den ganzen Block

Substitution-Permutation-Netzwerke (SPN)

- Input in kleine Blöcke teilen (z. B. 8 Bit)
- auf jeden kleinen Block eine (**zufällige**) Substitution anwenden
- Ausgabebits **mischen** (Permutation)
- „von vorne“ — mit **anderen** S_i's pro Runde



ⓘ Ausführlicher (mit durchgerechnetem Mini-Beispiel)

SPN – Erklärung Schritt für Schritt

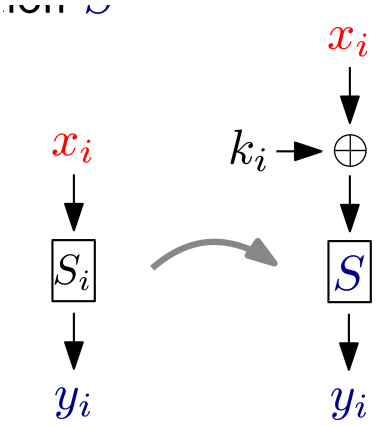
Iterative Verschlüsselung & S-Boxen

- Statt $y_i := S_i(x_i)$ mit rundenabhängigen S_i nutzt man in der Praxis **eine feste Funktion** S (in AES: $S : \{0,1\}^8 \to \{0,1\}^8$) und setzt

$$y_i := S(x_i \oplus k_i)$$

- **Iterative encryption:** Verschlüsselung läuft in **Runden**; Runde i wendet eine **invertierbare** Funktion $f_{k_i} : \{0,1\}^\ell \to \{0,1\}^\ell$ an, wobei der **Subkey** k_i aus k abgeleitet wird (*key schedule*)

- **S-Box** (Substitution box): zentrale Komponente, liefert **Confusion** (Nicht-Linearität) — ohne sie wäre die ganze Chiffre linear und damit trivial angreifbar



DES — Data Encryption Standard (1977)

- **Blockgröße 64 Bit**, **Schlüssellänge 56 Bit**
- Historischer Ablauf:
  - motiviert durch **kommerzielle** Anwendungen
  - **1972:** Ausschreibung durch NBS (→ NIST)
  - **1974:** Kandidaten, u. a. **Lucifer (IBM, Feistel)**
  - **NSA „half“**: Keylänge von **128 → 56 Bit** reduziert
  - **1977:** Standard veröffentlicht, aber die **Designkriterien blieben geheim**
  - Standard war **bis 1999** in Verwendung (3DES länger)

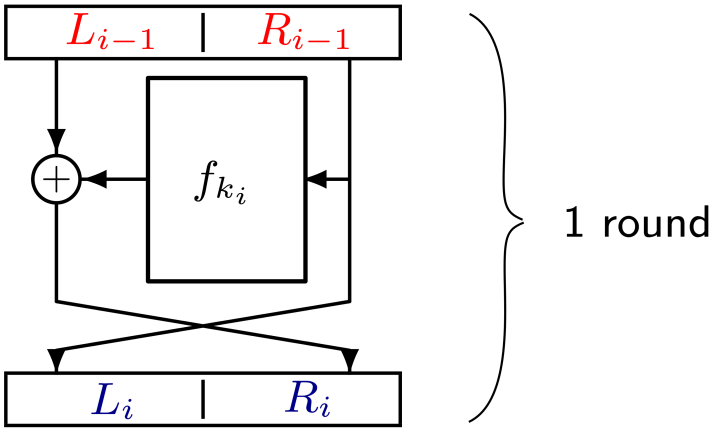
Feistel-Netzwerk

- Der 64-Bit-Block wird in zwei **32-Bit-Hälften** L_{i-1}, R_{i-1} geteilt; die Rundenfunktion f_{k_i} bekommt nur R_{i-1} (32 Bit) und liefert 32 Bit
- Rundenfunktion:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f_{k_i}(R_{i-1})$$

- **Muss f invertierbar sein? → Nein!** Das ist der ganze Trick eines Feistel-Netzwerks: nur die **Halbblock-Vertauschung** plus XOR macht die Runde umkehrbar, f selbst darf beliebig (z. B. eine S-Box-Kaskade) sein.
- **Entschlüsselung** (gleiche Struktur, Subkeys in umgekehrter Reihenfolge):

$$L_{i-1} = R_i \oplus f_{k_i}(L_i), \qquad R_{i-1} = L_i$$



> ⓘ **Ausführlicher (warum f nicht invertierbar sein muss)**
>
> Feistel-Netzwerk – Erklärung Schritt für Schritt

DES-Rundenfunktion

1. **Expansion** 32 → 48 Bit
2. **XOR** mit dem 48-Bit-Subkey k_i
3. **8 S-Boxen parallel** (6 → 4 Bit, also 48 → 32 Bit)
4. **Permutation** (32 Bit)

- DES: **16 Runden** ($f_1, \ldots, f_{16}$), key schedule: 56-Bit-Key → 16 Subkeys à 48 Bit
- Die S-Boxen sind **Lookup-Tabellen** — und genau sie tragen die Nicht-Linearität

Angriffe auf DES (§7.2.6)

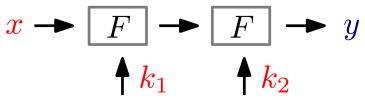
- **Differential cryptanalysis** (Biham & Shamir, 1990): unpraktisch, zeigt aber *theoretische* Schwäche; DES braucht 2^{47} Chiffrate zu gewählten Klartexten (**war IBM/NSA in den 70ern schon bekannt**)
- **Linear cryptanalysis** (Matsui, 1993): **Known-Plaintext**-Angriff, DES braucht 2^{43} Klartext/Chiffrat-Paare
- **Bester Angriff: exhaustive search** (Brute Force)
  - 1997: 69 Tage (1000e Rechner)
  - 1999: **Deep Crack** (ASICs): 22 Stunden
  - heute: **Minuten** (mit Preprocessing/FPGA)
- Fazit: **Design hielt den Attacken stand — der Schlüssel war zu kurz**

2DES und der Meet-in-the-Middle-Angriff

- Brute Force: gegeben x, y, finde k mit $F_k(x) = y$
- „Einfach zweimal verschlüsseln?“ → Keylänge **112 Bit** (zwei 56-Bit-Keys)

$$y = F_{k_2}\Big(F_{k_1}(x)\Big) \;\Rightarrow\; F_{k_1}(x) = F_{k_2}^{-1}(y)$$

- **Meet-in-the-middle:**
  1. Liste $F_{k_1}(x)$ für alle k_1 (Größe 2^{56})
  2. Berechne $F_{k_2}^{-1}(y)$ für alle k_2
  3. Match → Kandidat (k_1, k_2), per zweitem Paar verifizieren
- **Komplexität:** Speicher 2^{56}, Zeit 2^{57} DES-Evaluierungen ⇒ **2DES ist insecure** (nur Faktor 2 besser als DES!)

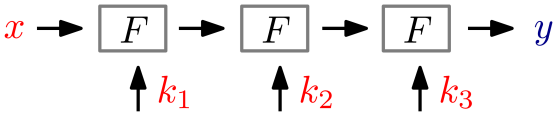


Triple DES (3DES)

- **Keys: 112 Bit**, Security: **112 Bit** (NIST bewertet konservativ: **80 Bit**)

$$y = DES_{k_1}\Big(DES_{k_2}^{-1}\Big(DES_{k_1}(x)\Big)\Big)$$

- **Warum nur zwei Keys?** Weil Meet-in-the-Middle bei drei unabhängigen Keys mit 2^{112} Aufwand sowieso möglich wäre (3 Keys à 56 Bit bringen keinen echten Zusatzgewinn bei dieser Struktur)
- **Warum DES^{-1} in der Mitte? Abwärtskompatibilität:** setzt man $k_1 = k_2$, wird daraus wieder einfaches DES
- Wird **noch immer** verwendet (Legacy-Systeme), gilt aber als veraltet



AES — Advanced Encryption Standard (Rijndael)

DES vs. AES im Direktvergleich

	DES	AES (Rijndael)
Struktur	S-P-Netzwerk, iteriert, **Feistel**	S-P-Netzwerk, iteriert (**kein** Feistel)
Blockgröße	64 Bit	**128 Bit**
Schlüssel	56 Bit	**128 / 192 / 256 Bit**
S-Boxen	**8 verschiedene**	**1** (für alle Bytes)
Rundenfunktion	**nicht** invertierbar	**invertierbar**
Optimiert für	**Hardware**	**byte-orientierte** Implementierungen
Design	**geheim** (NSA-Einfluss)	**offen** (NIST-Wettbewerb, Standard 2001)

AES-Parameter

- **Rijndael** (Joan **Daemen** & Vincent **Rijmen**), NIST-Wettbewerb, Standard seit **2001**
- Keylänge 128 / 192 / 256 Bit ⇒ **10 / 12 / 14 Runden**
- **Block = 16 Bytes = 128 Bit**, als 4×4-Matrix (**state**) geschrieben; Round Key ebenfalls 16 Bytes
- **Rundentransformation** besteht aus 4 Schritten:

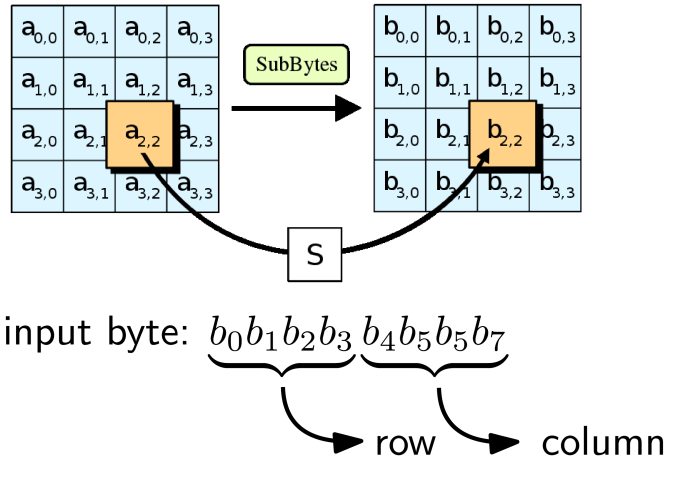
Schritt	Wirkung
Byte substitution (SubBytes)	Confusion
Shift rows	Diffusion
Mix columns	Diffusion
Round key addition (AddRoundKey)	Subkey einbringen

State-Layout & SubBytes

- Der 4×4-State hat die Einträge $a_{i,j}$ (**Zeile** i, **Spalte** j); der 16-Byte-Block $x_0, \ldots, x_{15}$ wird in der Anordnung der Folie hineingeschrieben:

$$\begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} = \begin{pmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{pmatrix}$$

- **Byte substitution (SubBytes):** die S-Box wird **byteweise** auf alle 16 Bytes angewandt: $b_{i,j} := S(a_{i,j})$
- Die S-Box ist eine **Lookup-Tabelle** (16×16): das Byte wird in zwei **Nibbles** zerlegt — die oberen 4 Bit $b_0 b_1 b_2 b_3$ sind die **Zeile**, die unteren 4 Bit $b_4 b_5 b_6 b_7$ die **Spalte** ⇒ der Tabelleneintrag ist das neue Byte
- Wirkung: **Confusion** — jedes Byte wird unabhängig **nicht-linear** ersetzt; *kein* Bit-Mixing zwischen Bytes (das machen ShiftRows/MixColumns)



AES S-box

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

AES-S-Box (vollständige Lookup-Tabelle, zum Nachschlagen – nicht auswendig)

Zeile = obere 4 Bit, Spalte = untere 4 Bit.

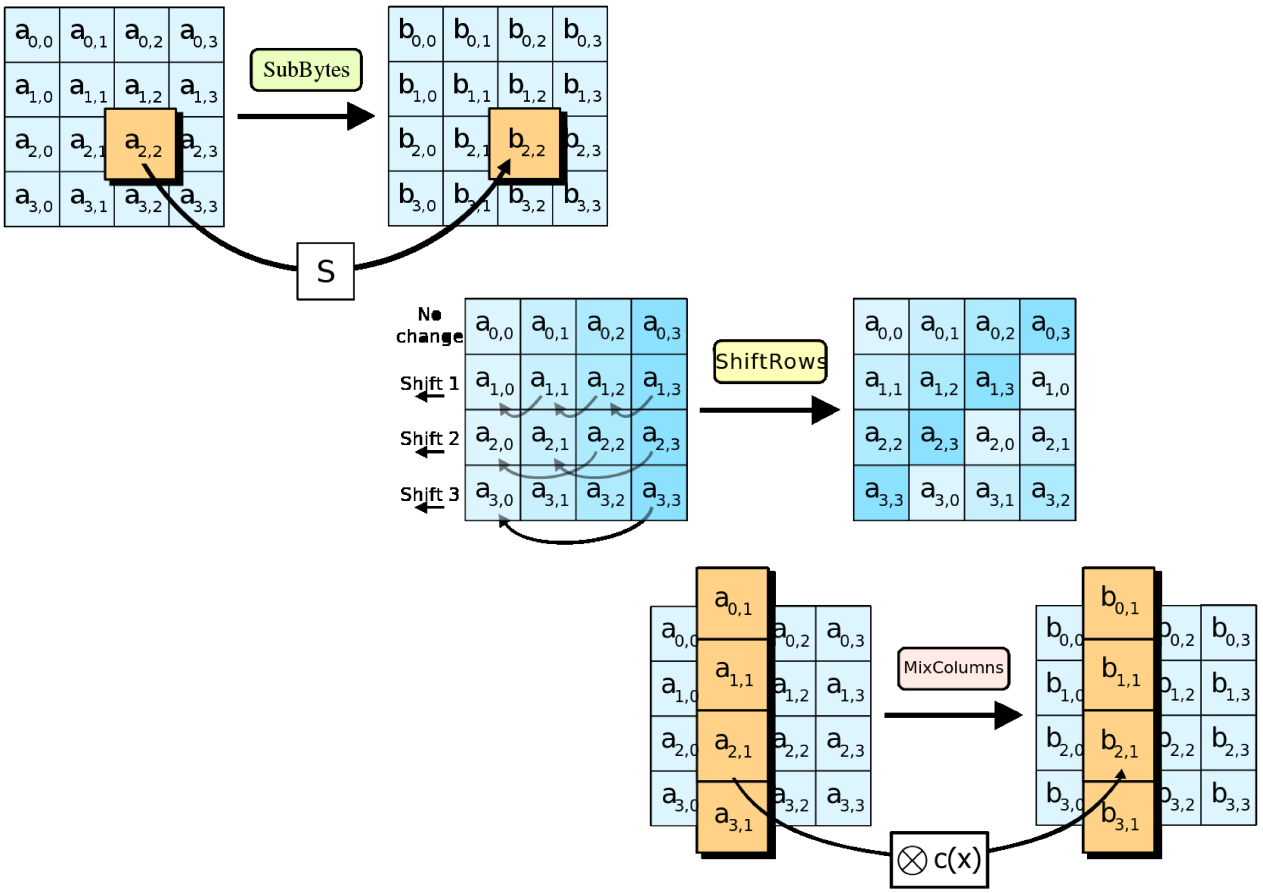
	00	01	02	03	04	05	06	07	08	09	0a	0b
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f

ShiftRows

- Zeile i des State wird um i Positionen **nach links** rotiert: Zeile 0 → 0, Zeile 1 → 1, Zeile 2 → 2, Zeile 3 → 3

$$\begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} \longrightarrow \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,1} & a_{1,2} & a_{1,3} & a_{1,0} \\ a_{2,2} & a_{2,3} & a_{2,0} & a_{2,1} \\ a_{3,3} & a_{3,0} & a_{3,1} & a_{3,2} \end{pmatrix}$$

- Zweck: **Diffusion über die Spalten hinweg** — vorher wirkt jede Spalte nur in sich, nach ShiftRows stammt jedes Byte einer Spalte aus einer **anderen** Zeile ⇒ erst jetzt kann MixColumns Bytes mischen, die ursprünglich weit auseinander lagen
- **SubBytes → ShiftRows → MixColumns** zusammen: nach **zwei** Runden hängt jedes Ausgabebyte von **allen 16** Eingabebytes ab (vollständige Diffusion)



Rijndael-S-Box — Algebra

- Bytes werden als Elemente von $\mathbb{F}_{2^8}$ interpretiert:

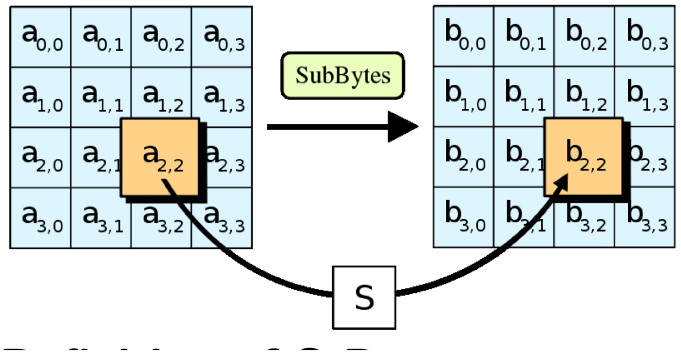
$$\mathbb{F}_{2^8} = \mathbb{Z}_2[X]/(X^8 + X^4 + X^3 + X + 1)$$

$$a = (a_0, \ldots, a_7) \mathrel{\widehat{=}} a_7X^7 + a_6X^6 + \cdots + a_1X + a_0$$

- Sei a' das **Inverse von** a in $\mathbb{F}_{2^8}$ (also $a \cdot a' = 1$; $a = 0 \mapsto 0$)
- Die S-Box ist dann eine **affine Transformation** über $\mathbb{Z}_2$:

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} a'_0 \\ a'_1 \\ a'_2 \\ a'_3 \\ a'_4 \\ a'_5 \\ a'_6 \\ a'_7 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

- Die Matrix A ist zyklisch aufgebaut (jede Zeile = Rotation der vorigen) — dadurch ist die S-Box **invertierbar** und **nicht-linear** (Inversion in $\mathbb{F}_{2^8}$ ist die einzige nicht-lineare Operation)
- Probe: $a = 0 \Rightarrow a' = 0 \Rightarrow b = c$, also $(b_0, \ldots, b_7) = (1, 1, 0, 0, 0, 1, 1, 0)$; mit b_0 als **niederwertigstem** Bit ist das 0x63 — genau der Eintrag links oben in der S-Box-Tabelle (Eingabe `00` ⇒ Ausgabe `63`)

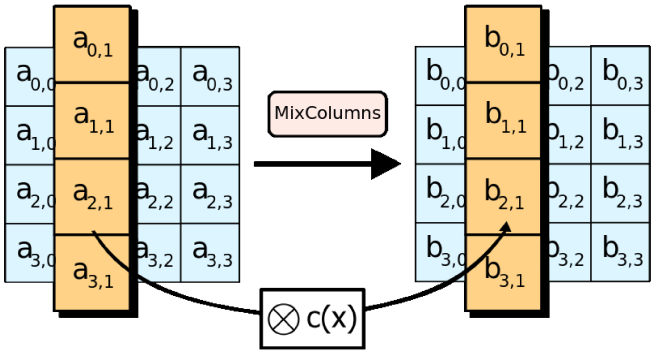


MixColumns

- Jede Spalte j wird mit einer festen Matrix über $\mathbb{F}_{2^8}$ multipliziert:

$$\begin{pmatrix} b_{0,j} \\ b_{1,j} \\ b_{2,j} \\ b_{3,j} \end{pmatrix} = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} a_{0,j} \\ a_{1,j} \\ a_{2,j} \\ a_{3,j} \end{pmatrix}, \qquad 0 \le j \le 3$$

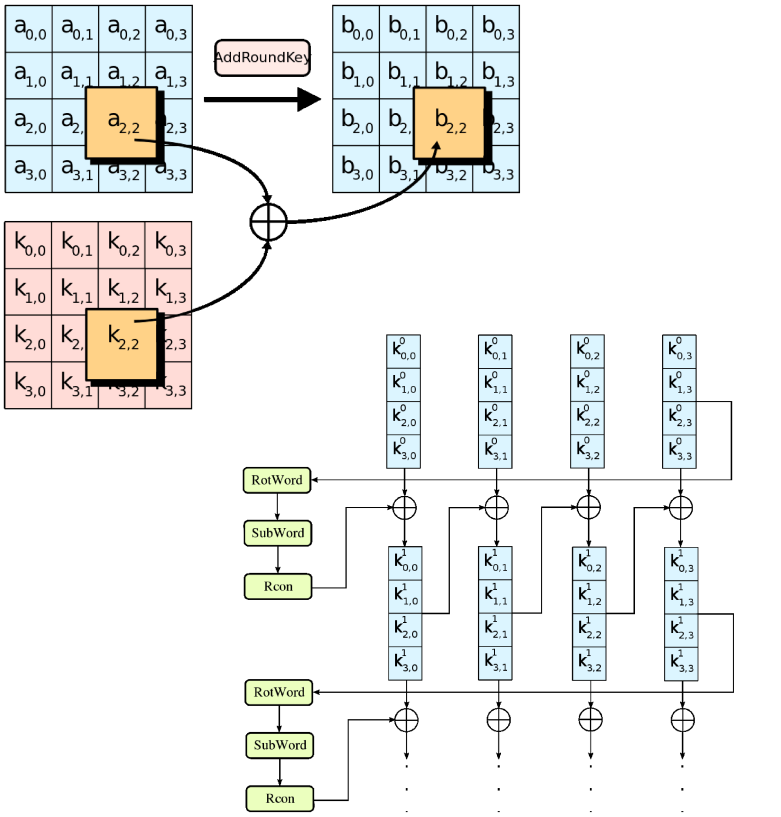
- Multiplikation mit 2 = Verschiebung + bedingtes XOR mit dem Reduktionspolynom; Multiplikation mit $3 = 2 \oplus 1$
- Zweck: **Diffusion innerhalb einer Spalte** — zusammen mit ShiftRows verteilt sich ein Bit über den ganzen 16-Byte-Block



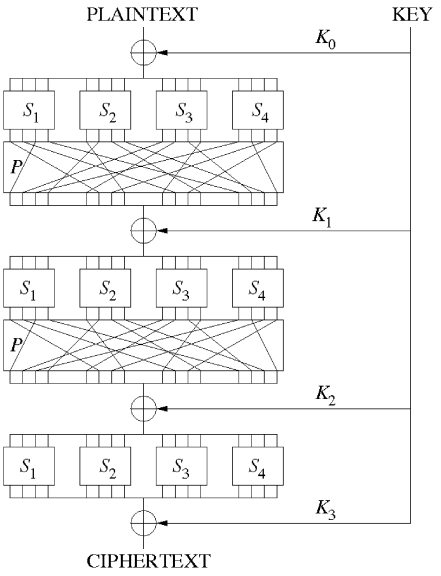
Round-Key-Addition & Key Schedule

- **AddRoundKey:** state $\oplus$ Round Key, byteweise und **einfaches XOR**: $b_{i,j} = a_{i,j} \oplus k_{i,j}$ — die einzige Rundenoperation, in die der Schlüssel eingeht
- **Gesamtstruktur** (AES-128): AddRoundKey mit K^0 → **9 volle Runden** (SubBytes, ShiftRows, MixColumns, AddRoundKey) → **letzte Runde ohne MixColumns** (nur SubBytes, ShiftRows, AddRoundKey) → Chiffrat; bei 192/256 Bit analog mit 12/14 Runden
- **Key Schedule** — die Round Keys $K^0, \ldots, K^{10}$ (je 16 Bytes = 4 Wörter $K_0^r..K_3^r$) werden aus dem Master-Key abgeleitet:
  - **Spalte 0** eines neuen Round Keys: **RotWord** (Wort rotieren, $[k_0, k_1, k_2, k_3] \to [k_1, k_2, k_3, k_0]$) → **SubWord** (S-Box auf jedes Byte) → XOR mit **Rcon** (Rundenkonstante $01, 02, 04, 08, 10, 20, 40, 80, 1\mathrm{B}, 36$ in $\mathbb{F}_{2^8}$) → XOR mit Spalte 0 des vorigen Round Keys

- **Spalten 1–3:** $K_j^r = K_j^{r-1} \oplus K_{j-1}^r$ (nur XOR, keine S-Box)
- ⇒ jedes Round-Key-Byte hängt von **allen** Master-Key-Bytes ab (im Gegensatz zu DES, wo ein Subkey-Bit nur aus einem Key-Bit kommt)



Derivation of round keys



Overall structure

20

Angriffe auf AES

- **2009:** Related-Key-Angriff mit Komplexität $2^{99.5}$ (nur theoretisch relevant, related-key ist kein realistisches Modell)
- **2011: Biclique-Angriff** (Meet-in-the-Middle) schneller als Brute Force: $2^{126.2}$ AES-Aufrufe für AES-128
- ⇒ **bester praktischer Angriff bleibt Brute Force** über den Key (2^{128}); AES gilt als sicher

✓ Kern für die Prüfung

- **Blockgröße ≥ 64 Bit** (Dictionary) und **Key ≥ 128 Bit** (Brute Force) — Begründung jeweils in einem Satz
- **Confusion = Substitution**, **Diffusion = Permutation**; S-Box liefert Nicht-Linearität
- **SPN vs. Feistel**: Feistel braucht **keine** invertierbare Rundenfunktion, Entschlüsselung mit umgekehrten Subkeys
- DES: 64/56, 16 Runden, 8 S-Boxen, Expansion 32→48; **schwache Keys, nicht schwaches Design**
- **Meet-in-the-Middle**: 2DES bricht auf 2^{57} (nicht 2^{112}!) — Zeit/Speicher-Trade-off hinschreiben können
- 3DES: EDE, 2 Keys, 112 Bit Security (NIST 80), $k_1 = k_2 \Rightarrow$ DES
- AES: 128-Bit-Block, 10/12/14 Runden, SubBytes/ShiftRows/MixColumns/AddRoundKey, S-Box = Inversion in $\mathbb{F}_{2^8}$ + affin
- AES-Details: S-Box ist eine 16×16-**Lookup-Tabelle** (obere 4 Bit = Zeile, untere = Spalte) · **ShiftRows** = Zeile i um i nach links (Diffusion über Spalten) · **Key Schedule** = RotWord + SubWord + Rcon, Spalten 1–3 nur XOR · **letzte Runde ohne MixColumns**

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025) — das T/F-Triplett zu Blockchiffren kam **komplett** dran:

- `1e` „Feistel bleibt invertierbar, auch wenn die Round Functions nicht invertierbar sind“ → **T**
- `1f` „Meet-in-the-Middle bricht 2DES in ≈ $\sqrt{\text{Brute-Force-Zeit}}$“ → **T** ($2^{112} \to 2^{57}$)
- `1g` „AES ist ein SPN“ → **T**

Sample 2026W

- `1b` „in einem SPN muss die Substitution invertierbar sein, sonst ist der SPN nicht invertierbar“ → **T** (der direkte Gegensatz zu `1e` /Feistel — beide Fragen zusammen sind die Standard-Falle)

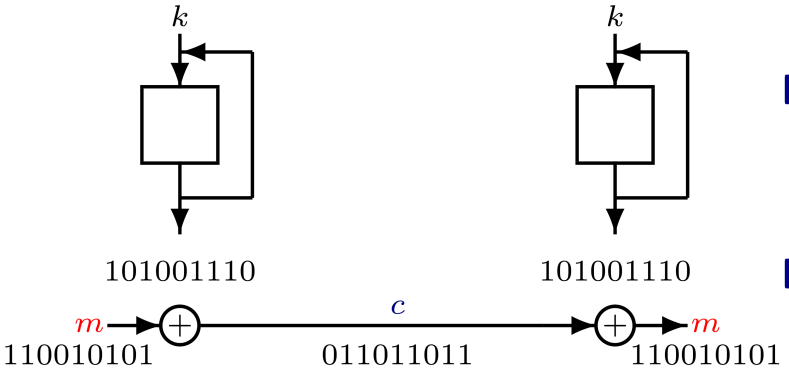
04 · Stream Ciphers & Computational Security

Stream Ciphers (§7.1)

- **Keystream generator:**
  - nimmt einen **zufälligen String (den Key)**
  - produziert einen **potenziell unendlichen** String, der *zufällig erscheint*
  - formal ist das genau ein **PRG** — die saubere Definition kommt in VO 5
- **Verschlüsselung wie OTP, aber:** der (lange) **echte Zufalls-Key** wird ersetzt durch einen **pseudorandom String**, generiert aus einem kurzen **Seed**

$$c_i = m_i \oplus s_i \qquad\qquad m_i = c_i \oplus s_i$$

- Korrektheit trivial, weil XOR selbstinvers ist; Sicherheit hängt **komplett** an der Qualität des Keystreams



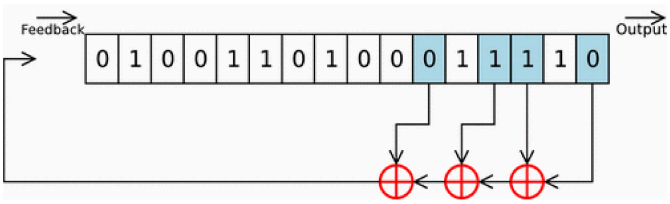
Encryption: $c_i = m_i \oplus s_i$

key stream

Decryption: $m_i = c_i \oplus s_i$

Linear-feedback shift register (LFSR)

- Schieberegister mit **linearer** Rückkopplung, sehr **effizient in Hardware**
- **linear ⇒ angreifbar** (die Linearität erlaubt es, den inneren Zustand aus wenigen Ausgabebits zu rekonstruieren)
- **Reale Konstruktionen:**
  - Kombination mehrerer LFSRs (z. B. **A5/1** in GSM)
  - **RC4** (verwendet in WEP)
  - Blockchiffre im *mode of operation* (→ CTR-Mode, VO 7)
- … **alle gebrochen** (A5/1, RC4/WEP, WEP insgesamt)



0 1 0 0 1 1 0 1 0 0 0 1 1 1 0
0 0 1 0 0 1 1 0 1 0 0 0 1 1 1
1 0 0 1 0 0 1 1 0 1 0 0 0 1 1
0 1 0 0 1 0 0 1 1 0 1 0 0 0 1
0 0 1 0 0 1 0 0 1 1 0 1 0 0 0
1 0 0 1 0 0 1 0 0 1 1 0 1 0 0 0 1

> ⚠ **Wichtig**
>
> Ein Stream Cipher ist **nur einmal** sicher: derselbe Keystream darf nie für zwei Nachrichten verwendet werden (sonst $c \oplus c' = m \oplus m'$ — genau der OTP-Key-Reuse-Angriff aus VO 2).

Recap & das Neue in VO 4 (§3.1)

> ⓘ **Schon in früheren Notizen erklärt**
>
> - **Dreiklang** Definitions / Assumptions / Security proof → VO 2
> - **Scheme-Syntax** ($Gen \to K$, $Enc : K \times M \to C$, $Dec : K \times C \to M$) + Korrektheit $Dec_k(Enc_k(m)) = m$ → VO 1
> - **Angreifermodell** in drei Achsen (Wissen / Ziel / Macht) + Kerckhoffs → VO 2

- **Neu in dieser VO:** die **Macht-Achse wird zur Leiter** der Angriffsklassen — jede Stufe ist strikt stärker als die vorige:

$$\text{ciphertext-only (eavesdropping)} < \text{known-plaintext} < \text{chosen-plaintext} < \text{chosen-ciphertext}$$

- **Security = Threat Model + Goal**: beides muss man explizit festlegen, sonst ist „sicher“ bedeutungslos
- Rest der VO: erst die Machtstufe *eavesdropper* sauber fassen (Ununterscheidbarkeit), dann von *perfect* auf *computational* umstellen

Indistinguishability — das Experiment

Experiment $PrivK^{\text{eav}}_{A,\Pi}$ für $\Pi = (Gen, Enc, Dec)$:

1. Challenger: $k \leftarrow Gen$; $b \leftarrow \{0,1\}$
2. A gibt zwei Nachrichten **gleicher Länge** $m_0, m_1 \in M$ aus
3. Challenger: $c \leftarrow Enc_k(m_b)$, schickt c an A
4. A gibt b' aus
5. Ausgabe $= 1$ **iff** $b = b'$ (sonst 0)

⇒ A muss **erraten**, welche der beiden Nachrichten verschlüsselt wurde.



$\mathcal{A}$

$m_0, m_1 \in \mathcal{M}$

c

b'



$k \leftarrow \text{Gen}$

$b \leftarrow \{0,1\}$

$c \leftarrow \text{Enc}_k(m_b)$

$\text{PrivK}^{\text{eav}}_{\mathcal{A},\Pi} = 1$ iff $b = b'$ (and 0 otherwise)

Def. 2.6 — perfect indistinguishability

Π ist *perfectly indistinguishable*, wenn für **jeden** Angreifer A gilt:

$$\Pr\left[PrivK^{\text{eav}}_{A,\Pi} = 1\right] = \tfrac{1}{2}$$

($1/2$ = reines Raten; kein Angreifer kann besser sein.)

Lemma 2.7

Ein Schema ist **perfectly secret** (Def. 2.3: $Pr[M = m \mid C = c] = Pr[M = m]$) **genau dann**, wenn es **perfectly indistinguishable** (Def. 2.6) ist.
→ die „Ununterscheidbarkeits"-Formulierung ist zur Shannon-Definition **äquivalent**, aber leichter auf den computational Fall zu verallgemeinern.

Computational Security — warum man Perfect Secrecy aufgibt

Perfect secrecy erzwingt (Shannon):

- Key **so lang wie** die Nachricht
- Key darf nur **einmal** verwendet werden
  ⇒ **nicht praktisch** (Beweis: Thm. 2.11/2.12)

Brute-Force-Illustration (1 Key pro Takt):

- PC: $\approx 2^{32} \cdot 2^{25} = 2^{57}$ Keys pro Jahr
- Supercomputer: $\approx 2^{61} \cdot 2^{25} = 2^{86}$ pro Jahr
- … seit dem Urknall: $\approx 2^{86} \cdot 2^{33} = 2^{119}$

Computational security lockert die Anforderung an zwei Stellen:

1. Es werden nur **computationally bounded** (effiziente) Angreifer betrachtet
2. Der Angreifer darf mit **sehr kleiner Wahrscheinlichkeit** Erfolg haben

Warum 2^{-60} „sicher genug“ ist

Sicherheit versagt mit Wahrscheinlichkeit 2^{-60}: Wahrscheinlicher ist, dass den Absender ein **Blitz** trifft. Bei 1 Ereignis pro Sekunde müsste man 10^{11} Jahre warten.

Konkreter vs. asymptotischer Ansatz

Konkreter Ansatz

(t, ε)-indistinguishable

Für **alle** Angreifer A mit Laufzeit $\leq t$ gilt:

$$\Pr\left[PrivK^{\text{eav}}_{A,\Pi} = 1\right] \leq \tfrac{1}{2} + \varepsilon$$

- **+** relevant für reale Anwendungen (konkrete Zahlen!)
- **–** Sicherheit ist **nicht justierbar** (kein „Drehregler“)
- **– nicht robust**: In welchem Rechenmodell zählt man t? Turingmaschine? RAM? welche CPU, wie viele?

Asymptotischer Ansatz

- **Security Parameter $n \in \mathbb{N}$**, allen bekannt (denke: Keylänge; **größeres n ⇒ sicherer**)
- Laufzeiten (User **und** Angreifer) und Erfolgswahrscheinlichkeit sind **Funktionen von n**
- **Angreifer und User laufen in Zeit polynomial in n ⇒ p.p.t.** (*probabilistic polynomial time*): es gibt ein Polynom p, sodass die Laufzeit bei Inputlänge n höchstens $p(n)$ ist; Zufallswahlen sind erlaubt (wie bei Gen)

Warum Polynomialzeit das richtige Modell ist

Strong Church-Turing Thesis: jede physikalisch realisierbare Berechnung lässt sich auf einer Turingmaschine mit höchstens **polynomialem Slowdown** simulieren. ⇒ Alles, was „realistisch“ berechenbar ist, ist in (probabilistischer) Polynomialzeit.

Negligible — Def. 3.4

negligible

$f : \mathbb{N} \to \mathbb{R}^+$ heißt **negligible**, wenn für **jedes** positive Polynom p ein N existiert, sodass für alle $n > N$:

$$f(n) < \frac{1}{p(n)}$$

äquivalent: $f(n) < n^{-c}$ für jedes c und genügend große n.

- **Beispiele (alle negligible):** 2^{-n}, $2^{-\sqrt{n}}$, $n^{-\log n}$
- **Closure properties:**
  - $p(\cdot), q(\cdot)$ polynomiell ⇒ $p \cdot q$ und $p(q(\cdot))$ wieder polynomiell
  - $p(\cdot)$ polynomiell, $\mu(\cdot)$ negligible ⇒ $p \cdot \mu$ ist negligible
  - ⇒ „polynomiell viele Wiederholungen eines negligible-Ereignisses bleiben negligible“ (Union Bound)

Merksatz

Security holds except with probability negligible in n — das ist die Standardfloskel jeder computational-security-Definition.

Beispiel 3.3 — warum n erhöhen hilft

Angenommen:

- ehrliche Parteien: $10^6 \cdot n^2$ Zyklen
- Angreifer: nach $10^8 \cdot n^4$ Zyklen Erfolg mit W $\leq 2^{-n/2}$

Annahme	ehrliche Partei	Angreifer
2 GHz, $n = 80$	3,2 s	nach **3 Wochen**: Erfolg 2^{-40}
8 GHz, $n = 160$	wieder 3,2 s	nach **13 Wochen**: Erfolg 2^{-80}

⇒ **Asymmetrie:** die ehrliche Laufzeit wächst nur *polynomiell* (und bleibt bei Anpassung von n konstant), die Erfolgswahrscheinlichkeit des Angreifers fällt *exponentiell*. Das ist die eigentliche Aussage des asymptotischen Ansatzes.

Parametrisierte Schemata & Def. 3.7

- **Revised syntax:**

- $k \leftarrow Gen(1^n)$: Gen nimmt den Security Parameter in **unary**, liefert Key k mit $|k| \geq n$
- $c \leftarrow Enc_k(m)$: für $m \in \{0,1\}^*$ beliebiger Länge
- $m := Dec_k(c)$: Dec ist **o. B. d. A. deterministisch**

Def. 3.7 – private-key encryption scheme

Drei **probabilistic polynomial-time (p.p.t.)** Algorithmen (Gen, Enc, Dec) mit obiger Syntax und Korrektheit.

Computational Indistinguishability — Def. 3.8

Def. 3.8

Π ist **(computationally) indistinguishable in the presence of an eavesdropper**, wenn für **jeden p.p.t.** Angreifer A ein **negligible** $\varepsilon(\cdot)$ existiert mit

$$\Pr\left[PrivK^{\text{eav}}_{A,\Pi}(n) = 1\right] \leq \tfrac{1}{2} + \varepsilon(n)$$

- Experiment wie oben, aber: Input 1^n für A, $k \leftarrow Gen(1^n)$, und A muss m_0, m_1 mit $|m_0| = |m_1|$ wählen
- **Wichtig:** ε darf von A abhängen (Quantoren: $\forall$ p.p.t. A $\exists$ negl. ε), aber nicht von n bzw. der Stichprobe
- Das ist die **erste echte computational-Security-Definition** — und genau die, die in VO 5 (Thm. 3.16) mit dem PRG bewiesen wird

Plaintext Length

- Wir wollen Schemata für **beliebig lange** Nachrichten
- **Auch das OTP verrät die Länge** der Nachricht
- Verschlüsselung kann die Länge **prinzipiell nicht verbergen** (das Chiffrat muss ja mindestens so lang sein)
- ⇒ in der Definition **reflektiert** durch die Forderung $|m_0| = |m_1|$
- **Leaking der Länge kann ein Problem sein:**
  - Ja/Nein-Antworten
  - Database Search („wie viele Treffer?“)
- ⇒ mit **anderen Mitteln** behandeln (Padding, Length-Hiding, …)

Semantic Security

- Formalisierung von „das Chiffrat leakt **nichts** an einen Poly-Time-Angreifer außer mit negligible Wahrscheinlichkeit“ (Def. 3.12 in Katz-Lindell)

Theorem 3.13

Ein Private-Key-Schema ist **semantically secure** (in presence of an eavesdropper) **genau dann**, wenn es **computational indistinguishability** (eavesdropper) erfüllt.
→ man darf also immer mit der (handlicheren) Ununterscheidbarkeits-Definition arbeiten.

Offene Frage der Folie

„What guarantees does indistinguishability yield?“ → Antwort im Kern: **nichts** über den Plaintext ist vorhersagbar; jede *beliebige* Funktion des Plaintexts, die A aus dem Chiffrat berechnen kann, kann er auch **ohne** Chiffrat berechnen (bis auf negligible Fehler). Das ist die inhaltliche Bedeutung von Semantic Security.

✓ Kern für die Prüfung

- Stream Cipher: $c_i = m_i \oplus s_i$, Keystream aus kurzem Seed; **LFSR linear ⇒ angreifbar**, A5/1 & RC4 gebrochen
- Provable-Security-Dreiklang mit Beispielen nennen können
- **PrivK^eav-Experiment** auswendig skizzieren (Challenger b, m_0, m_1 gleicher Länge, b', Ausgabe $b = b'$)
- Def. 2.6 $\leftrightarrow$ Lemma 2.7 (perfect secrecy $\Leftrightarrow$ perfect indistinguishability)
- Die **zwei Lockerungen**: bounded adversary + kleine Fehlerwahrscheinlichkeit
- **Concrete vs. asymptotic**: Vor-/Nachteile, (t, ε) vs. p.p.t. + negligible
- **Def. 3.4 negligible** + Beispiele (2^{-n}, $2^{-\sqrt{n}}$, $n^{-\log n}$) + Closure (poly × negl = negl)
- **Def. 3.8** aufschreiben können; ε darf von A abhängen
- Warum $|m_0| = |m_1|$ gefordert wird (Länge ist nicht verbergbar)
- Thm. 3.13: semantic security $\Leftrightarrow$ computational indistinguishability

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025)

- `1h` „f negligible ⇏ $n^{|f(n)|}$ negligible“ → **F** (Gegenbeispiel $f(n) = 2^{-n}$: $n^{2^{-n}} \to 1$)

Sample 2026W

- `1c` „Stream Ciphers müssen den Keystream **komplex nicht-linear** mit dem Plaintext mischen“ → **F** (Mischen ist reines XOR; die Nicht-Linearität steckt im **PRG**)
- `1d` „f negligible ⇏ $f(n)^n$ negligible“ → **T** (für große n ist $f(n) \leq 1$, also $f^n \leq f$; *nicht* verwechseln mit der 2025W-Variante $n^{|f(n)|}$ → **F**)
- `1e` „es gibt EAV-sichere Schemes für $\{0,1\}^*$ auch **ohne** $|m_0| = |m_1|$“ → **F** (Ciphertext-Länge ist nicht verbergbar — genau der Punkt aus Plaintext Length)
- ⇏ Die zwei Negligible-Aussagen (`1h` 2025W vs. `1d` Sample) sind **absichtlich** die gleiche Falle mit vertauschter Antwort: Exponent vs. Faktor lesen!

05 · Pseudorandomness & Security Proofs

Was ist „zufällig“? (§3.3.1)

- Welcher Bitstring ist uniform zufällig: `0000000000000000` oder `0110110010110110`?
- **Keiner von beiden „mehr“** — bei uniformer Ziehung aus $\{0,1\}^{16}$ hat jeder String Wahrscheinlichkeit 2^{-16}
- **Randomness ist eine Eigenschaft von Verteilungen**, nicht von einzelnen Strings:

$$D : \{0,1\}^n \to [0,1], \qquad \sum_{x \in \{0,1\}^n} D(x) = 1$$

- **Uniformverteilung:**

$$U_n : \{0,1\}^n \to [0,1], \qquad x \mapsto 2^{-n}$$

Pseudorandomness

- Idee: **„nicht von uniform unterscheidbar“**
- Notation: $x \leftarrow D$ heißt „sample x gemäß D“
- **Historisch:** D galt als pseudorandom, wenn es **statistische Tests** T_i besteht:

$$\Pr_{x \leftarrow D}\Big[T_i(x) = 1\Big] \approx \Pr_{x \leftarrow U_n}\Big[T_i(x) = 1\Big] \quad \text{für } i = 1, \dots$$

- **Kryptographie:** man weiß **nicht**, welchen Test der Angreifer verwendet ⇒ es müssen **alle effizienten Tests** bestehen

> **Warum „effizient“ essenziell ist**
>
> Ein **unbeschränkter** Test könnte einfach nachsehen, ob y im **Bild** von G liegt — das Bild hat nur 2^n Elemente im Raum der Größe $2^{\ell(n)}$. Dieser Test unterscheidet **perfekt** (Vorteil ≈ 1). „Alle effizienten Tests“ ist also die **stärkste Forderung, die überhaupt erfüllbar ist**.

Pseudorandomness — konkret

- Sei $D : \{0,1\}^n \to [0,1]$ eine Verteilung
- D ist (t, ε)**-pseudorandom**, wenn für alle A mit Laufzeit $\leq t$:

$$\Big|\Pr_{x \leftarrow D}\Big[A(x) = 1\Big] - \Pr_{x \leftarrow U_n}\Big[A(x) = 1\Big]\Big| \leq \varepsilon$$

- (t, ε) als „Drehregler“ statt p.p.t. ist derselbe Schnitt wie in VO 4: konkreter Ansatz

Pseudorandomness — asymptotisch

- Sequenz von Verteilungen $\{D_n\}_{n\in\mathbb{N}}$ mit $D_n : \{0,1\}^{\ell(n)} \to [0,1]$, ℓ polynomiell
- $\{D_n\}$ ist **pseudorandom**, wenn für **alle p.p.t.** A ein **negligibles** $\varepsilon(\cdot)$ existiert mit

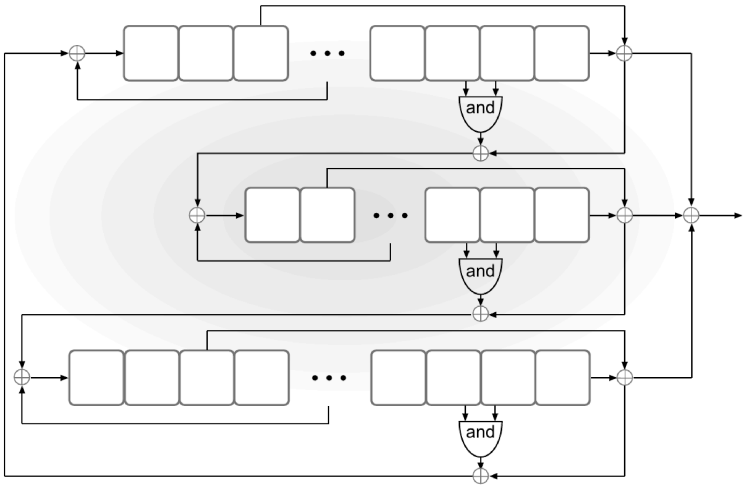
$$\left| \Pr_{x\leftarrow D_n}\left[A(x)=1\right] - \Pr_{x\leftarrow U_{\ell(n)}}\left[A(x)=1\right] \right| \le \varepsilon(n)$$

- n = **Security Parameter**

> ⚠ **Quantoren-Reihenfolge**
>
> $\forall$ p.p.t. A $\exists$ negl. ε: A wird **zuerst** gewählt, ε darf **von** A (also von dessen Laufzeit) abhängen, aber **nicht** von der gezogenen Stichprobe. Falsch wäre $\exists\varepsilon\,\forall A$ — dann wäre die Schranke fix und ein stärkerer Gegner dürfte sie überschreiten.

Pseudorandom Generators (PRGs)

- Ein PRG ist ein **deterministischer** Poly-Zeit-Algorithmus, der
  - einen uniformen **Seed** nimmt und
  - ihn in einen **längeren pseudorandom Output** expandiert
- **Motivation:** aus „wenigen“ echten Zufallsbits → „viele“ pseudorandom Bits
- Beispiel aus der Praxis: **Trivium** (hardware-optimierte Stromchiffre, eSTREAM-Portfolio) mit 80-Bit Seed → bis zu 2^{64} Bit Keystream; der Keystream **ist** der PRG-Output



Trivium

$$\{0,1\}^{80} \to \{0,1\}^{2^{64}}$$

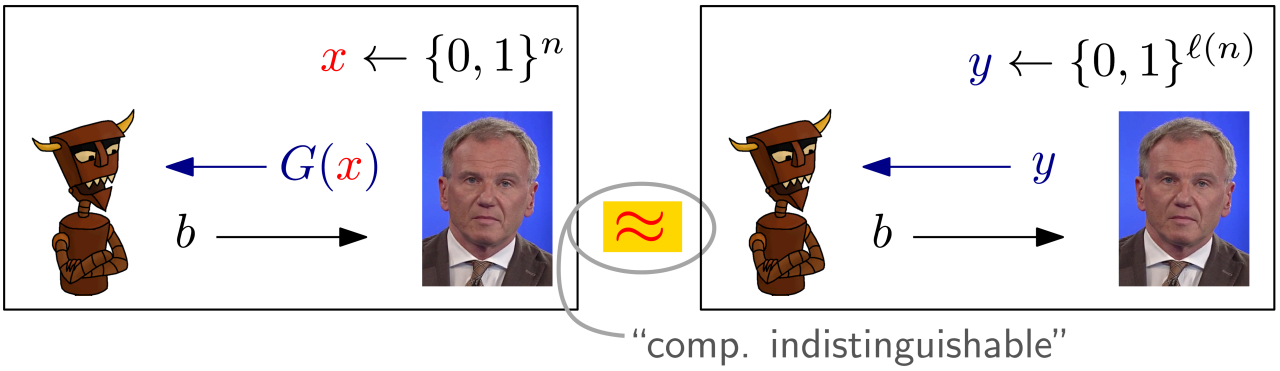
Definition 3.14 — PRG

Ein (deterministischer) Poly-Zeit-Algorithmus G mit $|G(x)| = \ell(|x|)$ für alle x ist ein **Pseudorandom Generator**, wenn er

1. **expanding** ist: $\ell(n) > n$ für alle n, **und**
2. **pseudorandom** ist: $\{G(U_n)\}_{n\in\mathbb{N}}$ ist eine pseudorandom Sequenz von Verteilungen, d. h. für alle p.p.t. A existiert negligible ε:

$$\left| \Pr_{x\leftarrow U_n}\left[A(G(x)) = 1\right] - \Pr_{y\leftarrow U_{\ell(n)}}\left[A(y) = 1\right] \right| \leq \varepsilon(n)$$

- G definiert also die Verteilungssequenz $G(U_1), G(U_2), \ldots$
- **Bild:** links $x \leftarrow \{0,1\}^n \to y := G(x)$; rechts $y \leftarrow \{0,1\}^{\ell(n)}$; ein Unterscheider A mit Ausgabe b darf die beiden nicht auseinanderhalten („**comp. indistinguishable**“)
- Die Expansion darf **minimal** sein: schon $\ell(n) = n + 1$ qualifiziert; lange Pads erzeugt man durch **Iteration** bzw. Einspeisen von (Seed, Zähler)



for all p.p.t. $\mathcal{A}$ there exists negligible $\varepsilon(\cdot)$ s.t.

$$\left| \Pr_{x\leftarrow U_n}\left[\mathcal{A}(G(x)) = 1\right] - \Pr_{x\leftarrow U_{\ell(n)}}\left[\mathcal{A}(x) = 1\right] \right| \leq \varepsilon(n)$$

Existieren PRGs?

- **Wir wissen es nicht.** Aber: $\exists$ PRG $\Rightarrow P \neq NP$
  (Wäre $P = NP$, dann wäre auch der „ist y im Bild von G?“-Test effizient lösbar ⇒ perfekter Unterscheider ⇒ kein PRG)
- **PRGs lassen sich aus schwachen Annahmen konstruieren:**

$$\text{PRGs} \Leftarrow \text{one-way functions} \quad (\text{z. B. Faktorisierungsproblem})$$

- Beispiel: $x = (p, q) \mapsto y := N = p \cdot q$ ist **leicht**, der Umweg (Faktorisieren) ist **hart**

OWF ⇔ PRG

Das ist keine Einbahnstraße: PRG ⇒ OWF ist direkt klar (G ist schwer invertierbar), und OWF ⇒ PRG ist das **HILL-Theorem** (Hardcore-Bit nach Goldreich–Levin + daraus gebauter PRG). „Schwache Annahme“ heißt also: **irgendeine Einwegfunktion existiert** — das ist die schwächste Standardannahme der symmetrischen Krypto.

- **Plan der Vorlesung:**
  - bestimmte Konstruktionen **als PRG annehmen**
  - dann mit PRGs Verschlüsselung bauen, die die **Schwächen perfekt sicherer Schemata** (kurzer Key, einmalige Nutzung) überwindet

Sicherheitsbeweise: Proofs by Reduction (§3.3.2)

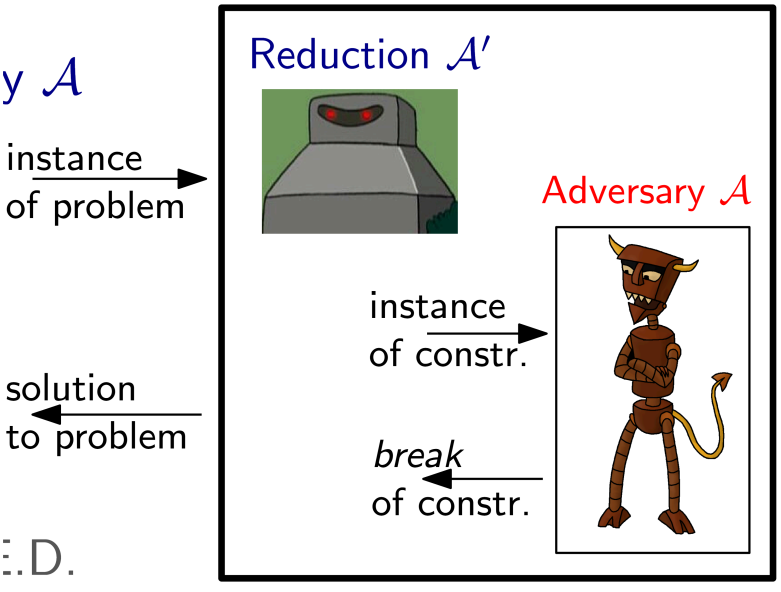
- Ziel: **„Problem hart ⇒ Konstruktion nicht brechbar“**
- **Annahme:** das Problem ist hart
- **Annahme (Widerspruch):** es gibt einen effizienten Angreifer A gegen die Konstruktion
- **Konstruktion:** ein Algorithmus A' gegen das Problem, der A verwendet

$$\underbrace{\text{Instanz des Problems}}_{\text{hart}} \longrightarrow \boxed{A' \text{ (Reduction)}}$$

$$\longleftrightarrow$$

$$\boxed{A \text{ (Angreifer)}} \longleftarrow \text{Instanz der Konstruktion}$$

- A' bekommt eine Problem-Instanz, simuliert für A eine Konstruktions-Instanz, und nutzt A's Antwort als Lösung
- ⇒ **kein solches A existiert** (Q.E.D.)



Wo die eigentliche Arbeit steckt

A' muss A **exakt die View simulieren**, die A im echten Angriff sieht — sonst ist A's Erfolgswahrscheinlichkeit nicht übertragbar. Weicht die Simulation nur negligibly ab, gilt: bricht A die Konstruktion mit Vorteil δ, dann löst A' das Problem mit Vorteil $\geq \delta - \mathrm{negl.}$; da das Problem hart ist, muss δ negligible sein. Zusätzlich: A p.p.t. $\Rightarrow A'$ p.p.t. (Poly-Overhead).

Das Pseudo-One-Time-Pad (§3.3.3)

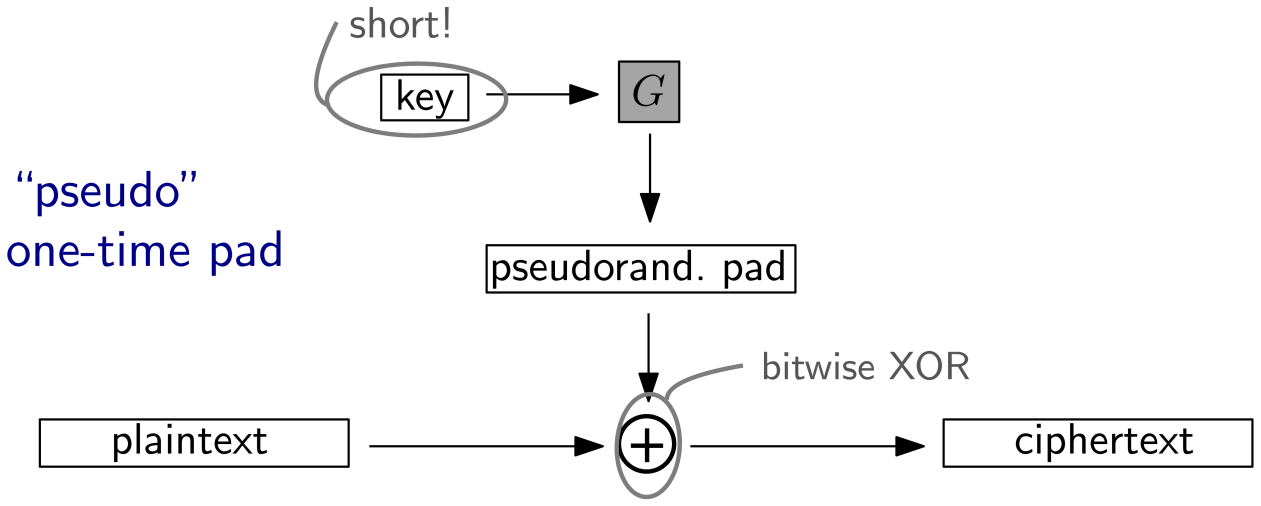
- Problem des OTP: Key **so lang wie die Nachricht** (OTP in VO 2)
- Idee: **PRG als Pad-Generator** — der kurze Seed wird zum langen Pad *expandiert*

Construction 3.17

Sei G ein PRG mit $|G(k)| = \ell(|k|)$.

- $Gen(1^n)$: $k \leftarrow \{0,1\}^n$; return k (Security Parameter $= n$)
- $Enc_k(m)$: return $c := G(k) \oplus m$ ⇒ $M = \{0,1\}^{\ell(n)}$
- $Dec_k(c)$: return $m := G(k) \oplus c$

Key nur n Bit, Pad aber $\ell(n)$ Bit; Entschlüsselung ist gratis, weil XOR selbstinvers ist: $G(k) \oplus c = m$.



Preis für den kurzen Key

Der Pad ist **nicht** echt zufällig ⇒ er darf nur **einmal** verwendet werden. Bei zwei Nachrichten kürzt sich $G(k)$ im XOR weg (siehe letzte Folie).

Beweisziel

Theorem 3.16

Ist G ein PRG, dann ist das Pseudo-OTP (Construction 3.17) **computationally indistinguishable in the presence of an eavesdropper** (Def. 3.8):

$$\Pr\left[PrivK^{\text{eav}}_{A,\Pi}(n) = 1\right] \leq \tfrac{1}{2} + \varepsilon(n)$$

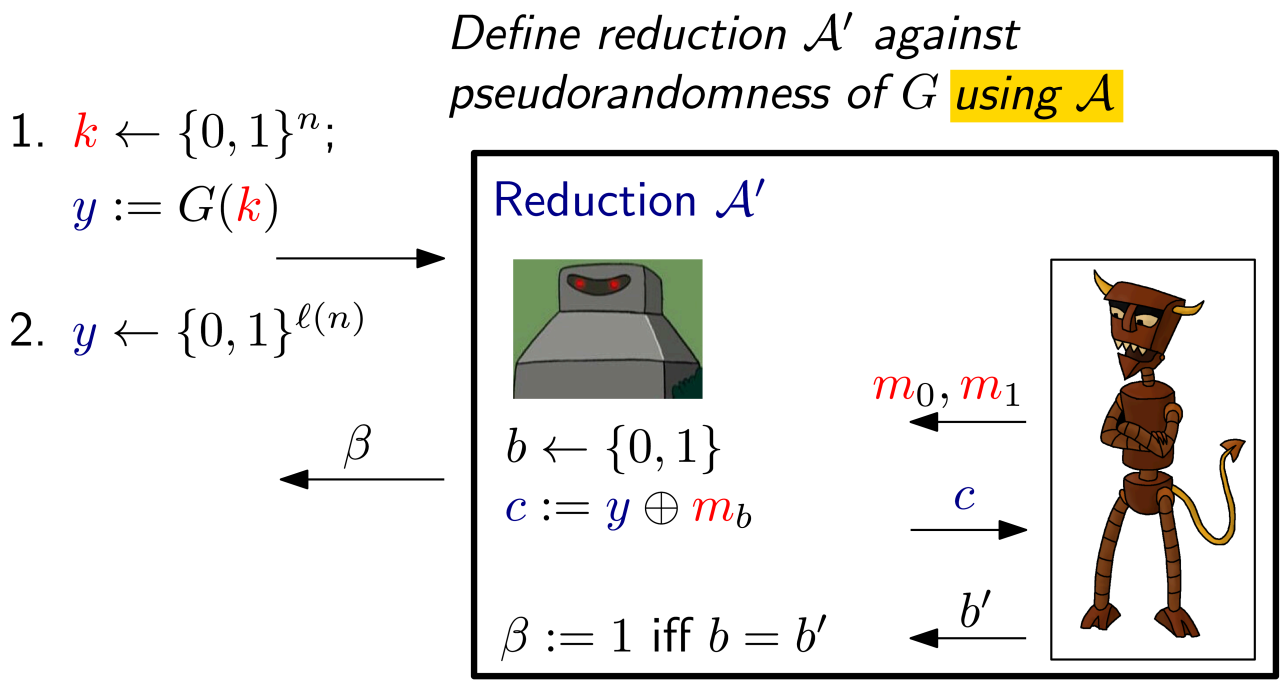
- Erinnerung Def. 3.8: für jeden p.p.t. A existiert negligible $\varepsilon(\cdot)$, sodass $\Pr[PrivK^{\text{eav}}_{A,\Pi}(n) = 1] \leq 1/2 + \varepsilon(n)$
- Das $1/2$ ist die **Rate-Basislinie**: rein zufälliges Raten von b gewinnt immer mit $1/2$

Beweis — Schritt 1: Reduction A' gegen die Pseudorandomness von G

- A' bekommt einen String y (entweder $y = G(k)$ mit $k \leftarrow \{0,1\}^n$ **oder** $y \leftarrow \{0,1\}^{\ell(n)}$) — das ist die **Herausforderung** des PRG-Spiels
- A' simuliert damit den PrivK-Challenger für A:

1. A gibt m_0, m_1 aus
2. $b \leftarrow \{0,1\}$
3. $c := y \oplus m_b$ ← **das ist die einzige Stelle, wo y verwendet wird**
4. A antwortet b'
5. A' gibt $\beta := 1$ aus **iff** $b = b'$

- Da A p.p.t. ist, ist auch A' p.p.t.



Beweis — Schritt 2: die zwei Fälle

Fall 1: $y = G(k)$, $k \leftarrow \{0,1\}^n$

- Dann ist $c = G(k) \oplus m_b = Enc_k(m_b)$ — A' simuliert den **echten** PrivK-Angriff **exakt**

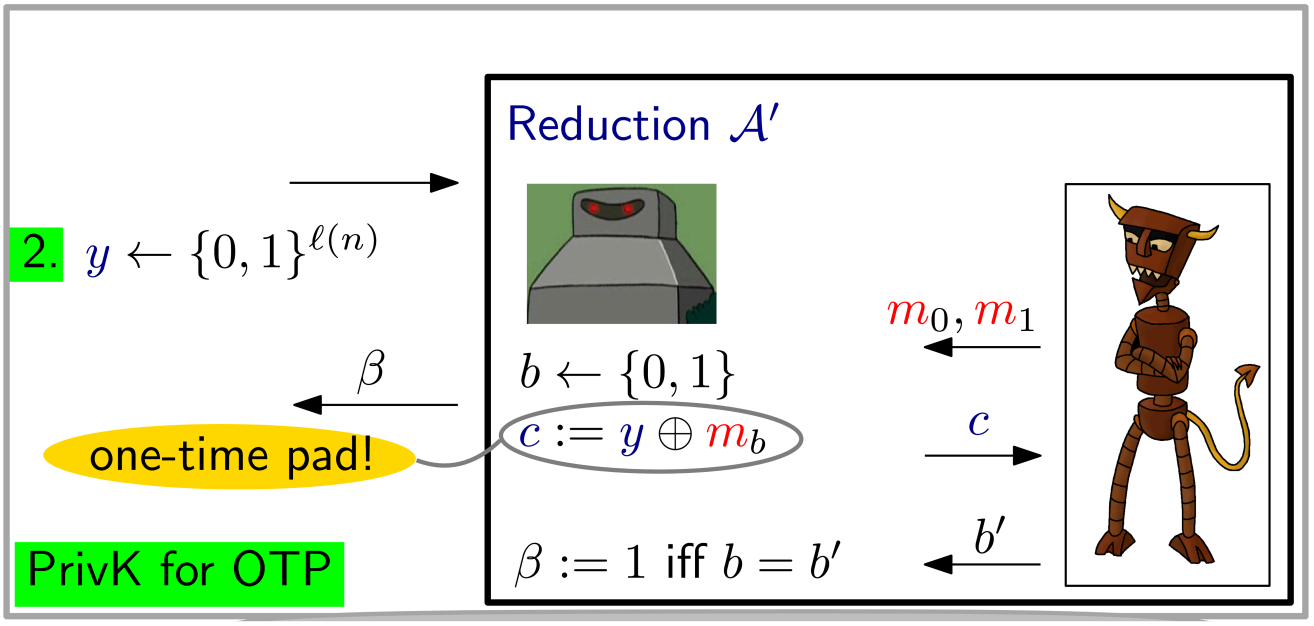
$$\Pr_{k \leftarrow U_n}\left[A'(G(k)) = 1\right] \;=\; \Pr\left[PrivK^{\text{eav}}_{A,\Pi}(n) = 1\right]$$

Fall 2: $y \leftarrow \{0,1\}^{\ell(n)}$ uniform

- Dann ist $c = y \oplus m_b$ ein **echtes One-Time-Pad** ⇒ perfekt sicher

$$\Pr_{y \leftarrow U_{\ell(n)}}\left[A'(y) = 1\right] \;=\; \tfrac{1}{2}$$

(für jede Wahl von m_0, m_1 — genau die Aussage von Perfect Secrecy)



Beweis — Schritt 3: zusammensetzen

- **PRG-Eigenschaft** (angewendet auf A'): es gibt negligible ε mit

$$\left|\Pr_{k \leftarrow U_n}\left[A'(G(k)) = 1\right] - \Pr_{y \leftarrow U_{\ell(n)}}\left[A'(y) = 1\right]\right| \le \varepsilon(n)$$

- Einsetzen der beiden Fälle:

$$\left|\Pr\left[PrivK^{\text{eav}}_{A,\Pi}(n) = 1\right] - \tfrac{1}{2}\right| \le \varepsilon(n)$$

$$\Rightarrow\; \Pr\left[PrivK^{\text{eav}}_{A,\Pi}(n) = 1\right] \le \tfrac{1}{2} + \varepsilon(n)$$

- ⇒ Π erfüllt **computational indistinguishability** (Def. 3.8) ∎

Beweismuster merken

Ein Beweis per Reduction hat immer dieselbe Form: **(1)** Annahme (G ist PRG), **(2)** Widerspruchsannahme (es gibt A gegen Π), **(3)** baue A' gegen G, **(4)** simuliere A's View, **(5)** zeige: Erfolg von A' = Erfolg von A im einen Fall, $= 1/2$ im anderen, **(6)** Widerspruch zur Annahme.

Grenzen des Ergebnisses

- **Annahme:** Was, wenn G **nicht** pseudorandom ist? → dann bricht der Beweis (und tatsächlich das Schema)
- **Definition:** Was, wenn die Definition **nicht stark genug** ist? → z. B. nur Eavesdropper (kein CPA/CCA), nur ein Chiffrat
- **Key-Reuse:** Das Schema ist ein **One-Time**-Pad — einmalig!

Key-Reuse-Angriff (Two-Time-Pad)

- derselbe Mechanismus wie beim OTP (VO 2), nur mit $G(k)$ statt eines echten Pads:

$$c = G(k) \oplus m, \quad c' = G(k) \oplus m' \;\Rightarrow\; c \oplus c' = m \oplus m'$$

- $G(k)$ kürzt sich heraus ⇛ die **Klartext-Differenz** ist bekannt (**Two-Time-Pad**); bei bekanntem oder geratenem Format ist die Kombination (m, m') oft eindeutig („buy/sell“ vs. „yes/no“)
- **Konsequenz:** pro Nachricht frischer Key bzw. **frische Nonce** (so macht es der CTR-Mode, → VO 7); denselben Keystream **nie zweimal** verwenden

✓ Kern für die Prüfung

- **Randomness ist eine Eigenschaft der Verteilung**, nicht des Strings; $U_n(x) = 2^{-n}$
- Warum „**alle effizienten** Tests“: Bild-von-G-Test macht unbeschränkte Unterscheidung trivial
- **Def. 3.14** vollständig: expanding ($\ell(n) > n$) + pseudorandom (Ungleichung mit $G(U_n)$ vs. $U_{\ell(n)}$)
- $\exists$ PRG $\Rightarrow P \neq NP$; PRG $\Leftarrow$ OWF (und tatsächlich $\Leftrightarrow$, HILL)
- **Reduction-Schema** in 4 Schritten erklären + „A' muss View exakt simulieren“
- **Construction 3.17** aufschreiben, Korrektheit, und **Thm. 3.16** beweisen können — inkl. der zwei Fälle ($G(k)$ ⇛ echtes PrivK-Spiel, uniformes y ⇛ OTP ⇛ $1/2$)
- Grenzen: Annahme, Definition, **Key-Reuse** ($c \oplus c' = m \oplus m'$)

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025)

- `3)` **große Frage (6 P.)** Pseudo-OTP mit $G : \{0,1\}^n \to \{0,1\}^{2n}$ und Parity-Trick: $r := k_{p(m)}$, $c := m \oplus r \oplus (p(r) \| 0^{n-1})$
  - a) **Korrektheit**: $p\Big(r \oplus (p(r) \| 0^{n-1})\Big) = 0$ ⇛ $p(c) = p(m)$ ⇛ `Dec` wählt dasselbe k_i und das zweite XOR hebt sich weg
  - b) **nicht EAV-sicher**: weil $p(c) = p(m)$ gilt, **leakt das Chiffrat die Parity des Klartexts** ⇛ A wählt $m_0 = 0^n$, $m_1 = 0^{n-1}1$ und liest b direkt aus $p(c^*)$ (Vorteil 1)
  - ⇛ Kernlehre: **PRG-Output ist nur dann ein OTP, wenn der Key pro Nachricht frisch/uniform ist** (Construction 3.17) — ein 1-Bit-„Münzwurf“ $p(m)$ ist keine Randomisierung

Sample 2026W

- `1f` „ein PRG ist ein **deterministischer** Algorithmus“ → **T**
- `1g` „$G'(x) := G(x) \| G(x)$ ist ein PRG, wenn G eines ist“ → **F** (Distinguisher: prüfe, ob beide Hälften gleich sind — bei G' immer wahr, bei uniform mit 2^{-2n}; die Verdopplung ist eine *deterministische Relation* im Output)

06 · PRFs & CPA-secure Encryption

Chosen-Plaintext Attacks (CPA) (§3.4.2)

> ⓘ **Schon erklärt**
>
> Kerckhoffs, die **Ziel**-Achse und die Leiter der Angriffsklassen stehen in VO 2 bzw. VO 4.

- **Neu: die Macht-Achse steigt eine Stufe** — der Angreifer darf Klartexte **selbst wählen** und erhält die zugehörigen Chiffrate (*chosen-plaintext*)
- **Warum ist das realistisch?** Man verschlüsselt nie nur eine geheime Nachricht: wer den Sender „füttern“ kann (gewählter Cookie, gewählter Request, gewählte Mail), kennt Paare $(m, Enc_k(m))$ — für ihn ist CPA das *richtige* (nicht zu starke) Modell
- **Single- vs. multi-message:** $PrivK^{\text{eav}}$ verschlüsselt genau **eine** Nachricht, real schickt Alice aber $m_1, m_2, \ldots$ (Folie 3-1 / 3-2). Statt das Experiment umzubauen, definiert man die **stärkere** Notion (CPA) — und Thm. 3.23 sagt, dass sie Multi-Message automatisch mit abdeckt

CPA-Security (Definition 3.21)

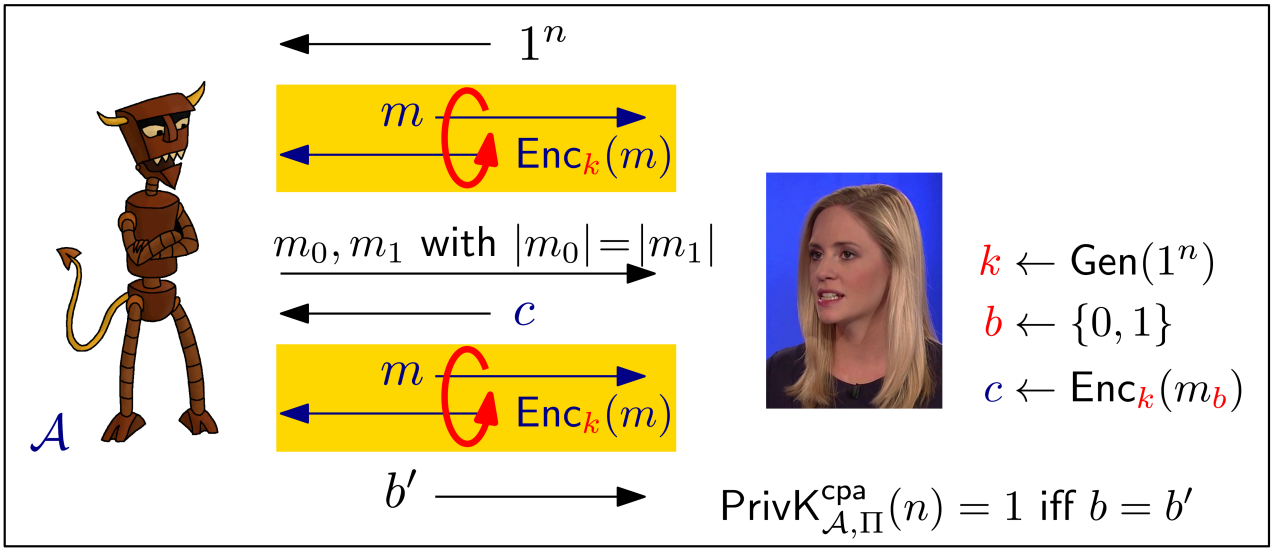
Das Experiment $\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi}(n)$:

1. $k \leftarrow \mathrm{Gen}(1^n)$, $b \leftarrow \{0,1\}$
2. A darf beliebig oft $\mathrm{Enc}_k(m)$ anfragen (chosen plaintexts)
3. A sendet m_0, m_1 mit $|m_0| = |m_1|$
4. Challenger sendet $c \leftarrow \mathrm{Enc}_k(m_b)$
5. A antwortet b'; gewinnt iff $b' = b$

$$\Pi \text{ ist CPA-sicher} \iff \forall \text{ p.p.t. } A: \ \Pr[\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi}(n) = 1] \le \tfrac{1}{2} + \varepsilon(n)$$

- **Multi-message security** (Thm. 3.23): CPA-sicher für eine Nachricht ⇒ CPA-sicher für viele (Hybrid-Argument)
- **Deterministische Schemes können nicht CPA-sicher sein:** A wählt m_0, m_1, erhält c^*, fragt m_0 nochmal an → $c^* = c_1$ verrät b. ⇒ **nur randomisierte** Schemes können CPA-sicher sein.

$\text{PrivK}^{\text{cpa}}_{\mathcal{A},\Pi}(n)$ for $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$



Pseudorandom Functions (PRF) (§3.5.1)

Random Functions

- $\text{Func}_n := \{f : \{0,1\}^n \to \{0,1\}^n\}$; eine zufällige Funktion ist eine uniforme Wahl aus Func_n
- Eine Funktion aufzuschreiben kostet $n \cdot 2^n$ Bit (exponentiell) ⇒ man kann sie dem Angreifer nicht als ganze Tabelle geben

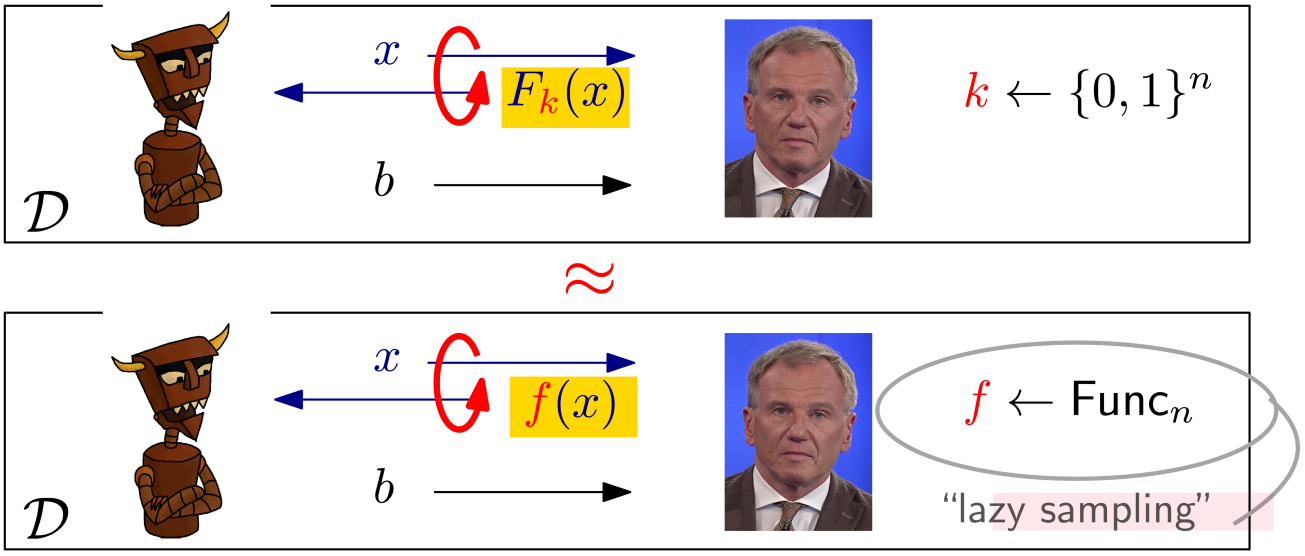
Keyed Functions & PRF

- **Keyed function:** $F : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}^*$; Key k definiert $F_k : x \mapsto F(k,x)$
- **Length-preserving:** $|F(k,x)| = |k| = |x|$ (Key-Länge = Input-Länge = Output-Länge)
- **PRF:** effiziente (poly-Zeit) keyed function, für uniformes k: F_k ist von einer uniformen $f \leftarrow \text{Func}_n$ **ununterscheidbar**

Definition 3.24 (Oracle-Indistinguishability): für alle p.p.t. D existiert negl. ε:

$$\left| \Pr_{k \leftarrow \{0,1\}^n} \left[D^{F_k(\cdot)}(1^n) = 1 \right] - \Pr_{f \leftarrow \text{Func}_n} \left[D^{f(\cdot)}(1^n) = 1 \right] \right| \leq \varepsilon(n)$$

- Der Angreifer hat **Oracle-Zugriff** (fragt x an, erhält $F_k(x)$), nicht die ganze Tabelle — wegen der exponentiellen Größe
- **Lazy Sampling:** man wählt $f(x)$ erst dann, wenn x angefragt wird; solange kein x wiederholt wird, sind die Antworten unabhängig uniform — wie bei einer echten Zufallsfunktion



PRP (Pseudorandom Permutation)

- Permutation über $\{0,1\}^n$: $\mathrm{Perm}_n \subset \mathrm{Func}_n$
- Keyed Permutation: für jedes k ist F_k bijektiv und F_k^{-1} effizient berechenbar
- **PRP:** F_k (uniform k) ununterscheidbar von uniformem $f \leftarrow \mathrm{Perm}_n$
- **Block Ciphers** = praktische PRP-Konstruktionen (feste Blocklänge, Brute-Force kostet $\approx 2^n$)

CPA-secure Encryption aus PRFs (§3.5.2)

Erste Idee (PRP, deterministisch) — nicht CPA-sicher

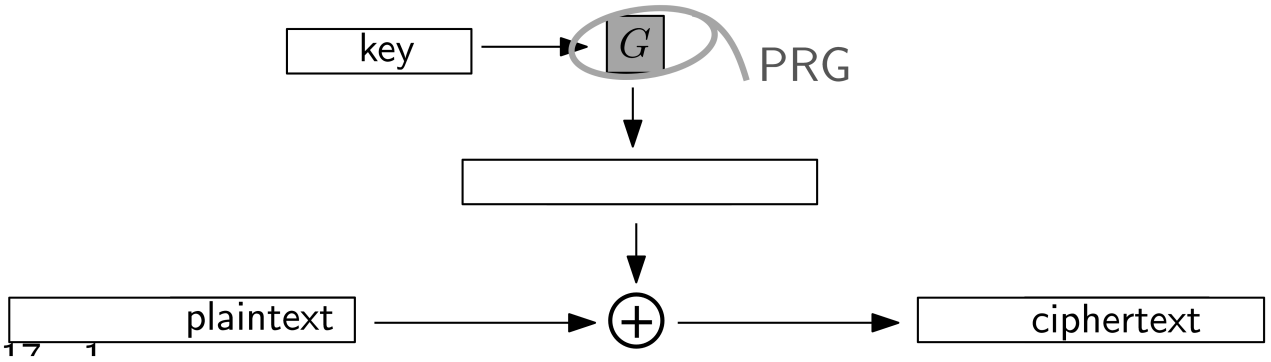
- $\mathrm{Enc}_k(m) := F_k(m)$, $\mathrm{Dec}_k(c) := F_k^{-1}(c)$ mit PRP F
- Passiv sicher, aber **nicht CPA-sicher** (deterministisch ⇒ gleicher Klartext ⇒ gleicher Ciphertext)

Construction 3.28 (probabilistisch)

Sei $F : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ eine PRF.

- $\mathrm{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$; return k
- $\mathrm{Enc}_k(m)$: $r \leftarrow \{0,1\}^n$; return $c := (r,\ F_k(r) \oplus m)$
- $\mathrm{Dec}_k((r,s))$: return $m := F_k(r) \oplus s$
- **r wird für jede Verschlüsselung frisch gewählt** ⇒ probabilistisch: gleicher Klartext ergibt verschiedene Ciphertexte
- Pseudo-One-Time-Pad mit PRF statt PRG als Pad-Generator

Recall: "pseudo" one-time pad



Beweis: Construction 3.28 ist CPA-sicher (Theorem 3.29)

Ziel: $\Pr[\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi}(n) = 1] \leq \frac{1}{2} + \varepsilon_0(n)$ für alle p.p.t. A.

Reduction D gegen die PRF F: nutzt A als Black-Box.

- D bekommt Oracle-Zugriff auf $F_k(\cdot)$ (oder $f(\cdot)$) und simuliert den CPA-Challenger für A:
  - Encrypt-Anfrage m: $r \leftarrow \{0,1\}^n$, $y := F_k(r)$, return $c := (r,\ y \oplus m)$
  - Challenge m_0, m_1: $r \leftarrow \{0,1\}^n$, $b \leftarrow \{0,1\}$, $y := F_k(r)$, return $c := (r,\ y \oplus m_b)$
  - A antwortet b'; D gibt $\beta := 1$ iff $b = b'$
- A p.p.t. ⇒ D p.p.t.

Zwei Fälle

Fall 1: $y = F_k(r)$ (echte PRF)

- D simuliert den echten PrivK-Angriff exakt

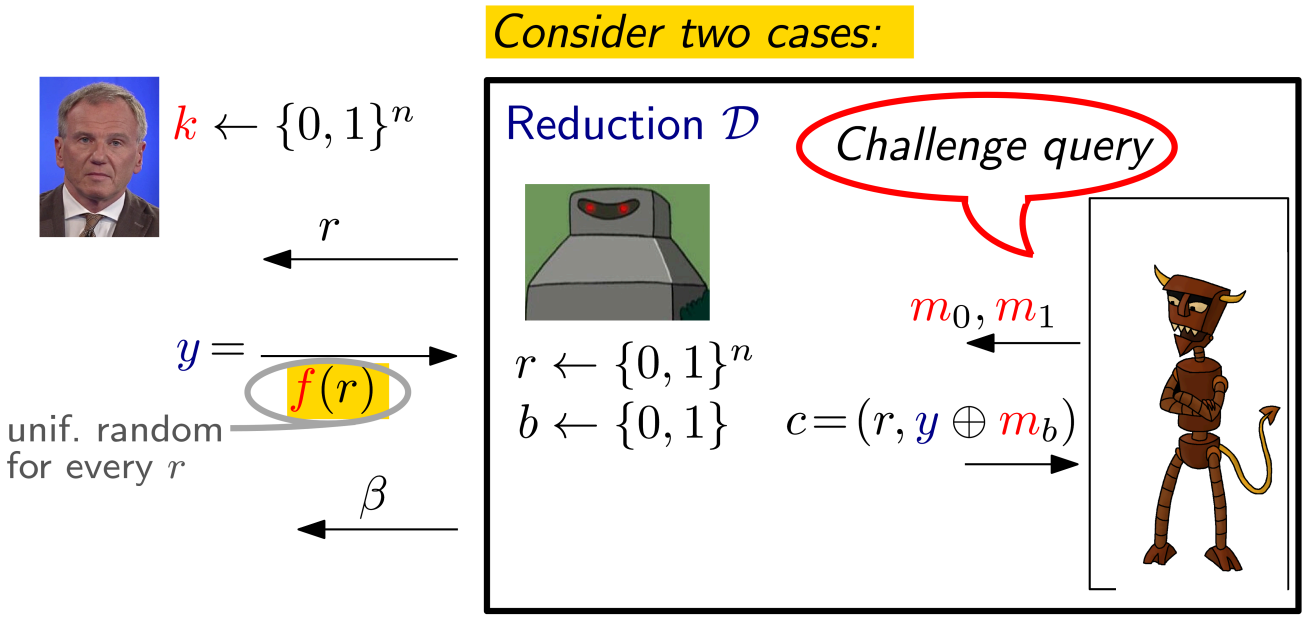
$$\Pr_k\left[D^{F_k}(1^n) = 1\right] = \Pr[\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi}(n) = 1]$$

Fall 2: $y = f(r)$ uniform zufällig (für jedes r)

- Dann ist $c = (r,\ f(r) \oplus m_b)$; da $f(r^*)$ (für den Challenge-r^*) **nirgends sonst** verwendet wird, ist $f(r^*) \oplus m_b$ uniform und unabhängig von m_b ⇒ A kann nicht besser als raten:

$$\Pr_f\left[D^f(1^n) = 1\right] = \tfrac{1}{2} \quad (\text{bis auf Repeat-Kollision})$$

Repeat-Event R: wenn der Challenge-r^* erneut als Anfrage-r auftritt, gilt $c \oplus c^* = m \oplus m_b$ ⇒ Information-Leak. Wahrscheinlichkeit $\leq q(n)/2^n$ (q Anfragen, 2^n mögliche r) — vernachlässigbar.



Zusammensetzen

PRF-Eigenschaft (angewendet auf D):

$$\left|\Pr_k D^{F_k} = 1 - \Pr_f D^f = 1\right| \leq \varepsilon(n)$$

Einsetzen:

$$\left|\Pr[\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi} = 1] - \tfrac{1}{2}\right| \leq \varepsilon(n) + \frac{q(n)}{2^n}$$

⇒ für alle p.p.t. A existiert negl. $\varepsilon_0(n) := \varepsilon(n) + q(n) \cdot 2^{-n}$ mit

$$\Pr[\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi}(n) = 1] \leq \tfrac{1}{2} + \varepsilon_0(n)$$

⇒ Π ist CPA-sicher ∎

> **Theorem 3.29**
>
> Ist F eine Pseudorandom Function, dann ist Construction 3.28 ein CPA-sicheres Private-Key-Encryption-Schema für Nachrichten der Länge n.

Proof-Idea (alternative Sicht)

- **Idealcpa-Game:** ersetze F_k durch uniformes $f \leftarrow \mathrm{Func}_n$ (lazy sampling)
- PRF-Sicherheit ⇒ reales und ideales Spiel sind ε-nah ($\Delta \leq \varepsilon(n)$)
- Im idealen Spiel gewinnt A nur, wenn r^* kollidiert (Repeat) ⇒ Vorteil $\leq q(n)/2^n$
- ⇒ $\Pr[\mathrm{PrivK}^{\mathrm{cpa}}_{A,\Pi} = 1] \leq \frac{1}{2} + \varepsilon(n) + \frac{q(n)}{2^n}$

✓ Kern für die Prüfung

- **CPA-Security (Def. 3.21):** chosen plaintexts + Challenge $m_0, m_1 \to c \leftarrow \mathrm{Enc}_k(m_b) \to b'$; Erfolg $\leq 1/2 + \mathrm{negl}$.
- **Deterministische Schemes** sind nie CPA-sicher (Repeat-Angriff)
- **PRF (Def. 3.24):** Oracle-Indistinguishability; Lazy Sampling
- **PRP:** Block Ciphers; F_k bijektiv + inverse effizient; PRP ist auch PRF (für $q \ll 2^{n/2}$)
- **Construction 3.28:** $c := (r,\ F_k(r) \oplus m)$, r frisch pro Verschlüsselung ⇒ probabilistisch, Expansion $|c| = |m| + n$
- **Beweis per Reduction:** D nutzt A als Black-Box; zwei Fälle (echte PRF ⇒ echtes Spiel; uniform ⇒ $1/2$); Repeat-Kollision vernachlässigbar
- **Theorem 3.29:** PRF ⇒ CPA-sicher (für Nachrichten der Länge n); längere Nachrichten → Modes of Operation

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025) — nur indirekt (CCA-Definition → VO 7), **kein** CPA-Beweis gefragt

Sample 2026W

- `2)` **große Frage (5 P.)** Lücken-Definition **PRF**: $\left|\Pr[D^{F_k(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]\right| \leq \varepsilon(n)$; links uniform $k \leftarrow \{0,1\}^n$, rechts uniform $f \leftarrow \mathrm{Func}_n$ (Menge **aller** Funktionen)
- `4)` **große Frage (11 P.)** CPA-Sicherheit anwenden: stateful CTR mit ctr **pro Aufruf nur um 1** erhöht
  - a) `Dec`: parse $c_0 \| c_1 \| \ldots \| c_\ell$, $m_i := c_i \oplus F_k(c_0 + i)$
  - b) **nicht CPA-sicher**, weil sich die Keystream-Blöcke zwischen zwei Aufrufen **überlappen** (ctr springt nur +1, nicht $+\ell$) ⇒ Keystream aus einer 0-Nachricht ablesen, Challenge entschlüsseln
  - c) für $m \in \{0,1\}^n$ (1 Block) **CPA-sicher**: pro Aufruf frischer Counter ⇒ Reduction auf PRF, im idealen Spiel ist $f(ctr)$ frisch uniform ⇒ OTP
  - ⇒ beliebtes Muster: **CPA-Beweis per Reduction** + Gegenbeispiel für längere Nachrichten

07 · Modes of Operation & CCA-Security

Wo wir stehen — zwei offene Baustellen

> ⓘ **Recap aus VO 6 (wird hier nicht wiederholt)**
>
> - **Angreifermodell** (Kerckhoffs, Ziele, Macht-Leiter) → VO 6
> - **CPA-Experiment** $\mathrm{PrivK}^{\mathrm{cpa}}$ + **Def. 3.21** → VO 6
> - **Construction 3.28** ($c := (r,\ F_k(r) \oplus m)$ mit frischem r) + **Thm. 3.29** inkl. Reduction-Beweis und Repeat-Event R → VO 6
> - **PRF / PRP** (Def. 3.24), Oracle-Zugriff, Lazy Sampling → VO 6

1. **Baustelle „lange Nachrichten“:** 3.28 verschlüsselt genau **einen** n-Bit-Block. Naiv blockweise verschlüsseln $(\mathrm{Enc}_k(m_1), \ldots, \mathrm{Enc}_k(m_t))$ ist zwar CPA-sicher (Multi-Message-Security, Thm. 3.23), kostet aber $|c| = 2|m|$ — jeder Block zahlt n Bit Zufall. Mit **Modes of Operation** kommt man auf $|c| = |m| + n$.
2. **Baustelle „Angreifer darf entschlüsseln“:** sobald A ein Decryption Oracle hat, ist 3.28 **malleable** und damit nicht mehr sicher → **CCA-Security**.

Modes of Operation (§3.6.3)

Ziel: Nachrichten länger als n Bits mit einem Blockcipher / PRP F verschlüsseln.

- **ECB** (Electronic Code Book) — *deterministisch, kaputt*
- **CBC** (Cipher Block Chaining) — *braucht PRP + IV + Padding*
- **CTR** (Counter Mode) — *braucht nur PRF, expansion-optimal*

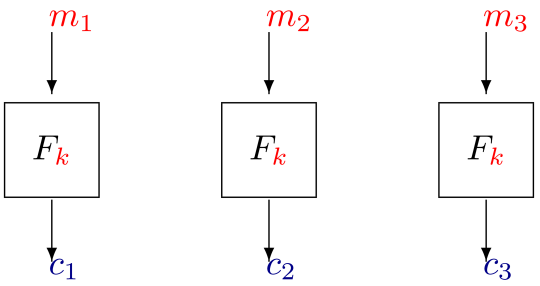
ECB — nicht sicher!

$$\mathrm{Enc}_k(m_1, \ldots, m_t) = F_k(m_1), \ldots, F_k(m_t)$$

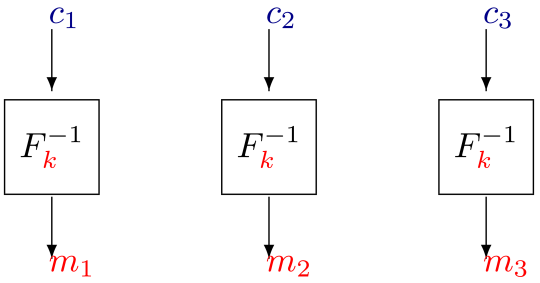
- **Deterministisch** ⇒ nicht CPA-sicher (gleicher Klartext ⇒ gleicher Ciphertext)
- **Problem:** wenn $m_i = m_j$, dann $c_i = c_j$ ⇒ **Muster erscheinen im Ciphertext**
- ⇒ nicht einmal EAV-sicher (passiver Angreifer reicht)
- **Do not use!**

ECB mode

Encryption:



Decryption:

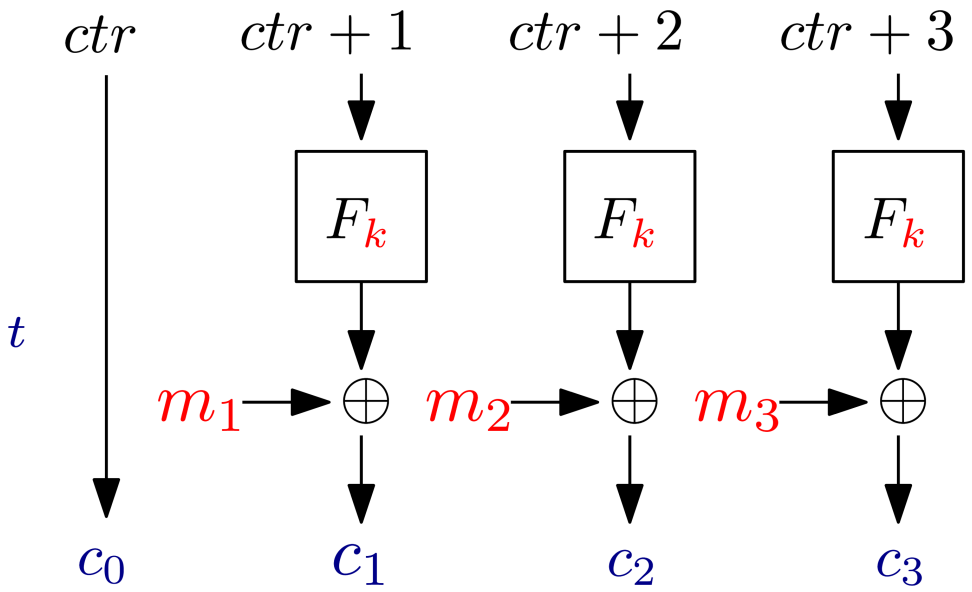


10

CTR (Counter Mode)

$$c_i := F_k(\mathrm{ctr} + i) \oplus m_i, \quad c_0 := \mathrm{ctr}$$

- $\mathrm{ctr} \leftarrow \{0,1\}^n$ frisch gewählt
- **Stream-Cipher-View:** $F_k(\mathrm{ctr} + i)$ ist der Keystream, XOR mit m_i
- **F muss nicht invertierbar sein** (PRF reicht) — im Gegensatz zu CBC, das F^{-1} für die Entschlüsselung braucht
- **Ciphertext expansion:** $|c| = |m| + n$ (optimal! — n Bits für den Counter)
- **Counter-Kollisionen sind katastrophal:** wenn $\mathrm{ctr} + i$ für zwei Nachrichten kollidiert, wird derselbe Keystream wiederverwendet (wie One-Time-Pad-Wiederverwendung) ⇒ Klartexte können XOR-t werden. Daher n groß wählen.



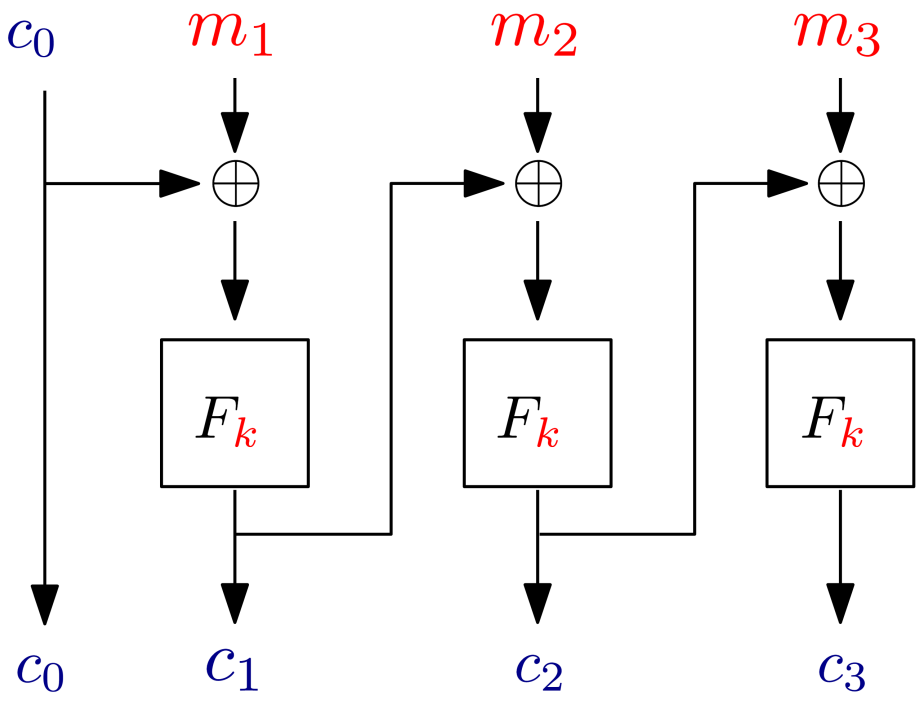
Theorem 3.33

Ist F eine Pseudorandom Function, dann ist CTR-Mode CPA-sicher.

CBC (Cipher Block Chaining)

$$c_i := F_k(m_i \oplus c_{i-1}), \quad c_0 \leftarrow \{0,1\}^n \text{ (IV)}$$

- Dec_k: $m_i := F_k^{-1}(c_i) \oplus c_{i-1}$ — Entschlüsselung ist **sequenziell** (Block i braucht c_{i-1})
- **IV** (Initialisierungsvektor) muss **frisch und unvorhersagbar** sein, sonst deterministisch ⇒ nicht CPA-sicher; IV darf **im Klartext** mitgeschickt werden
- **Padding nötig:** F verarbeitet ganze Blöcke ⇒ die Nachricht wird auf ein Vielfaches von n aufgefüllt (PKCS#7) — genau daraus wird später der **Padding-Oracle-Angriff**
- **Ciphertext expansion:** ein Block (der IV)
- **Wenn F eine PRP ist**, dann ist CBC CPA-sicher (Thm. 3.32)
- Verwendet in SSL/TLS (dort historisch mit genau den Padding-Problemen)



PRP vs. PRF

- **CBC** braucht PRP (inverse nötig für Decryption)
- **CTR** reicht PRF (nur Vorwärtsauswertung)

Vergleich der Modes (Klausur-Tabelle)

	ECB	CBC	CTR
Verschlüsselung	$F_k(m_i)$	$F_k(m_i \oplus c_{i-1})$	$F_k(\text{ctr}+i) \oplus m_i$
randomisiert?	**nein** (deterministisch)	ja (IV)	ja (Counter-Startwert)
CPA-sicher?	**nein**	ja, wenn F PRP (Thm. 3.32)	ja, wenn F PRF (Thm. 3.33)
Expansion	keine	$+n$ (IV)	$+n$ (Counter)
braucht F^{-1}?	ja (für Dec)	ja	**nein**
parallelisierbar?	ja	nur Verschlüsselung	**ja, auch Dec**
Padding nötig?	ja	ja	**nein**
Schwachstelle	Muster-Leak	Padding-Oracle, IV-Handling	Counter-/Nonce-**Reuse**

Chosen-Ciphertext Attacks (CCA) (§5.1)

Adversary-Power-Erweiterung

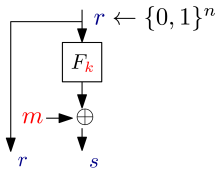
- Der Angreifer kann jetzt **auch entschlüsseln lassen** (Decryption Oracle), außer für die Challenge c^*
- **Warum realistisch?** Jede Reaktion auf ein manipuliertes Chiffrat ist Information: Fehlermeldung, Timeout, „Padding invalid“ — das ist ein Orakel (siehe Padding-Oracle unten)
- Experiment $\text{PrivK}^{\text{cca}}_{A,\Pi}(n)$:
  1. $k \leftarrow \text{Gen}(1^n)$, $b \leftarrow \{0,1\}$
  2. A darf $\text{Enc}_k(m)$ und $\text{Dec}_k(c)$ anfragen ($c \neq c^*$)
  3. A sendet m_0, m_1; Challenger sendet $c^* \leftarrow \text{Enc}_k(m_b)$
  4. A antwortet b'; gewinnt iff $b' = b$

> **Definition 5.1 — CCA-Security**
>
> Π ist **sicher gegen chosen-ciphertext attacks**, wenn für jeden p.p.t. A ein negligible $\varepsilon(\cdot)$ existiert mit
>
> $$\Pr\left[\text{PrivK}^{\text{cca}}_{A,\Pi}(n) = 1\right] \leq \tfrac{1}{2} + \varepsilon(n)$$
>
> Einziger Unterschied zu CPA (Def. 3.21): das **Decryption Oracle** ist zusätzlich verfügbar — verboten ist nur die Anfrage $c = c^*$.

Malleability-Angriff auf Construction 3.28

- Gegeben Challenge $(r, s := F_k(r) \oplus m_b)$
- Für beliebiges $\Delta \neq 0$: frage $\mathrm{Dec}_k(r,\ s \oplus \Delta)$ an
- Ergebnis: $F_k(r) \oplus (s \oplus \Delta) = m_b \oplus \Delta$ ⇒ b **leak**
- ⇒ CCA-sicheres Scheme darf nicht **malleable** sein: aus c darf man nicht c' mit bekannter Relation zum Klartext erzeugen können

> ⚠ **Malleability**
>
> Der Angreifer kann den Ciphertext manipulieren (hier: beliebige Bits flippen via XOR mit Δ), ohne den Key zu kennen. Deshalb braucht man **Authentizität** (MAC / AEAD).



Padding-Oracle Attack (warum CCA real ist)

- **Too paranoid? No!** Padding-Oracle-Angriff gegen SSL/TLS (PKCS#7-Padding im CBC-Mode)
- Der Angreifer muss **nur** lernen, ob die Entschlüsselung **erfolgreich** war (Padding gültig) — eine Fehlermeldung genügt
- Damit lässt sich der Klartext **byteweise** entschlüsseln: „nur CPA-sicher“ ist in der Praxis also wertlos
- **Keines** der Modes ECB/CBC/CTR ist CCA-sicher (alle sind **malleable**) ⇒ es fehlt **Authentizität** (MAC bzw. AEAD, z.B. Encrypt-then-MAC)

> 🔥 **Fazit der VO (und Ausblick)**
>
> CCA-Sicherheit ist die **stärkste sinnvolle** Machtstufe (mehr als „darf entschlüsseln“ kann ein Angreifer nicht). Kein reines Encryption-Scheme erreicht sie — deshalb braucht es **Integrität**: MACs & authenticated encryption (nächste VO).

✓ **Kern für die Prüfung**

- **ECB:** deterministisch, nicht CPA-sicher, Muster leaken — nie verwenden
- **CTR:** $c_i = F_k(\mathrm{ctr} + i) \oplus m_i$, $|c| = |m| + n$ (optimal), **PRF reicht**, Counter-/Nonce-Kollision katastrophal
- **CBC:** $c_i = F_k(m_i \oplus c_{i-1})$, **PRP nötig**, frischer IV, Padding, CPA-sicher (Thm. 3.32), in SSL/TLS
- **Vergleichstabelle** (randomisiert / CPA / Expansion / inverse nötig / Padding) aus dem Kopf hinschreiben
- **CCA-Sicherheit:** Angreifer darf auch entschlüsseln (außer c^*); Definition analog zu CPA, nur mit Dec-Oracle
- **Malleability:** Ciphertext ohne Key manipulierbar ⇒ CCA-sicher geht nur mit **Authentizität** (MAC/AEAD)
- **Padding-Oracle:** Angreifer lernt nur, *ob* die Entschlüsselung geklappt hat → reicht für Angriff
- $|c| = |m| + n$ **ist optimal:** mindestens n Zufallsbits sind für CPA-Sicherheit nötig (sonst ist das Scheme deterministisch)

⁝☰ Altprüfungen – was hierzu schon gefragt wurde

Midterm 2025W (03.12.2025)

- `1i` „ECB, CBC **und** CTR sind alle EAV-sicher“ → **F** (ECB ist deterministisch ⇒ Muster-Leak; A mit $m_0 = 0^n0^n$, $m_1 = 0^n1^n$ gewinnt mit Vorteil 1)
- `1j` „CTR ist sicher, solange der IV **verschieden** ist“ → **F** (IV muss **unvorhersehbar** sein — Theorem 3.33 fordert $\mathrm{ctr} \leftarrow \{0,1\}^n$ zufällig; bei Zähler-IV Keystream aus dem Enc-Oracle ablesen)
- `2)` **große Frage (5 P.)** Lücken-Definition **CCA**: $\Pr[\mathrm{PrivK}^{cca} = 1] \leq \frac{1}{2} + \varepsilon(n)$ · Schritt 2: Zugriff auf $Enc_k(\cdot)$ **und** $Dec_k(\cdot)$ · Schritt 3: $|m_0| = |m_1|$ · Schritt 5: darf c^* **nicht** an `Dec` übergeben · Schritt 6: $b' = b$
- `4)` **große Frage (6 P.) MAC** $Mac_k(m) = F_k(m) \oplus m$ per Reduction auf PRF als existentiell unforgeable zeigen
- `5)` **große Frage (3 P.) AE** konstruieren: Enc' = Encrypt-then-MAC, also $c \leftarrow Enc_{k_E}(m)$, $t \leftarrow Mac_{k_M}(c)$, Ausgabe (c, t); `Dec'` prüft erst $Vrfy_{k_M}(c, t)$

Sample 2026W

- `1h` „ECB ist mit einem PRP EAV-sicher“ → **F**
- `1i` „anders als CBC kann CTR mit einer **PRF** statt PRP instanziiert werden“ → **T**
- `1j` „jedes CCA-sichere Scheme ist auch CPA-sicher“ → **T** (CCA ist die stärkere Machtstufe ⇒ impliziert CPA; umgekehrt falsch)
- `4)` **große Frage (11 P.)** stateful CTR (Details → CTR in VO 7): ctr nur $+1$ pro Aufruf ⇒ Keystream-Reuse ⇒ **nicht** CPA-sicher

[!warning] Lücke in den Notes
MACs (2025W `4`) und Authenticated Encryption (2025W `5`) sind im Skriptum erst **nach** §5.1 (VO 8) — in diesen Notes nicht abgedeckt, kamen aber schon 6 + 3 Punkte dran. Wer die Midterm ernst nimmt: Encrypt-then-MAC + „PRF ⇒ sicherer MAC“ (Reduction, $Mac_k(m) = F_k(m) \oplus m$ ist bei F PRF sicher, weil m die Inputs nur verschiebt) nachlernen.

Zusatzmaterial

> **Info**
>
> Ab hier kommen nur noch zusätzliche Erklärungen die für das Verständnis einzelner Themen hilfreich sein könnten.

SPN_Explanation

1. Das Grundproblem, das ein SPN löst

Eine **Blockchiffre** ist im Kern:

- nimm einen Block fester Länge (z.B. 64 oder 128 Bit),
- mit einem geheimen Schlüssel k,
- verwandle ihn **umkehrbar** (bijektiv) in einen Ciphertext gleicher Länge.

Das Ziel: Der Ciphertext soll für einen Angreifer wie **reines Rauschen** aussehen. Ein **SPN (Substitution-Permutation Network)** ist ein besonders einfacher, aber sehr erfolgreicher Bauplan dafür.

Die Idee: Man hat **zwei Bausteine** — *Substitution* und *Permutation* — und wendet sie immer wieder abwechselnd an.

2. Die zwei Zutaten

2a. Substitution → „Konfusion"

Man zerlegt den Block in kleine Häppchen (z.B. 4-Bit- oder 8-Bit-Stücke) und ersetzt **jedes Häppchen** über eine feste Nachschlagetabelle, die **S-Box** (*Substitution box*):

S-Box = Nachschlagetabelle (hier für 2 Bit)

Eingabe	Ausgabe
00	01
01	11
10	00
11	10

Wichtig: die Abbildung ist **nichtlinear** und **bijektiv** (jeder Ausgabewert kommt genau einmal vor). Aus `11` wird `10`, aus `01` wird `11`, usw.

Die S-Box ist der **nichtlineare** Kern. Ohne sie wäre das ganze Verfahren linear und mit Gleichungssystemen knackbar.

2b. Permutation → „Diffusion"

Danach **vertauscht** man die Bit-Positionen im ganzen Block nach einer festen Regel (es wird *nichts* ersetzt, nur umsortiert):

Permutation = Bits umsortieren (4 Bit)

$(b_1, b_2, b_3, b_4) \to (b_3, b_1, b_4, b_2)$
`1011` → $(1, 0, 1, 1) \to (1, 1, 1, 0) = \texttt{1110}$.
Die Anzahl der Einsen bleibt gleich — es wandert nur ein Bit in ein anderes Häppchen.

So verteilt („diffuses") sich der Einfluss jedes Eingabebits über den ganzen Block.

3. Warum braucht man beides?

Jede Zutat allein ist nutzlos

- **Nur Substitution:** jedes Häppchen wird unabhängig verwandelt → man kann die Häppchen getrennt analysieren (wie eine Ersetzungschiffre in klein).
- **Nur Permutation:** komplett linear → als Gleichungssystem lösbar.

Zusammen in vielen Runden entsteht etwas, das wie eine Zufallsfunktion aussieht.

Das ist genau Shannons Prinzip (📖 IntroCrypto3, S. 5):

$$\text{Substitution} \Rightarrow \text{Konfusion} \qquad \text{Permutation} \Rightarrow \text{Diffusion}$$

4. Wie eine Runde aussieht

Ein SPN arbeitet **iterativ (in Runden)**. Eine Runde i macht **drei Schritte in dieser Reihenfolge**:

> ⓘ **Runde i**
>
> 1. **Key-Addition** K_i: Block **XOR** mit dem runden-spezifischen Subkey k_i.
> 2. **Substitution** S_i: jedes Häppchen über die S-Box ersetzen.
> 3. **Permutation** P_i: Bits umsortieren.

Als Formel:

$$x \xrightarrow{K_1} \xrightarrow{S_1} \xrightarrow{P_1} \xrightarrow{K_2} \xrightarrow{S_2} \xrightarrow{P_2} \dots \xrightarrow{K_r, S_r, P_r} C$$

Der S-Box-Schritt ist genau der von der Folie $y_i := S(x_i \oplus k_i)$ — zuerst Schlüssel einrühren, dann substituieren.

> **Wo steckt der Schlüssel?**
>
> **Nur** in der Key-Addition ($K_i(s) = s \oplus k_i$). S und P sind fest und **öffentlich**. Das ist für Q3 entscheidend.

5. Konkretes Mini-Beispiel (klein genug zum Nachrechnen)

4-Bit-Block, 2-Bit-Häppchen, S-Box aus 2a, Permutation $P(b_1, b_2, b_3, b_4) = (b_3, b_1, b_4, b_2)$.
Eingabe $x = 1011$, Rundenschlüssel $k_1 = 0110$, $k_2 = 1001$.

Runde 1:

Schritt	Rechnung	Ergebnis
Key-Add	$1011 \oplus 0110$	1101
Substitution	$[11][01] \to [10][11]$	1011
Permutation	$(1,0,1,1) \to (1,1,1,0)$	1110

Runde 2:

Schritt	Rechnung	Ergebnis
Key-Add	$1110 \oplus 1001$	0111
Substitution	$[01][11] \to [11][10]$	1110
Permutation	$(1,1,1,0) \to (1,1,0,1)$	1101

Ciphertext: $C = 1101$.

6. Entschlüsseln

Weil jeder Schritt umkehrbar ist, läuft das Ganze einfach **rückwärts**:

$$C \xrightarrow{P_r^{-1}} \xrightarrow{S_r^{-1}} \xrightarrow{K_r} \xrightarrow{P_{r-1}^{-1}} \dots \to x$$

- $K_i^{-1} = K_i$, denn XOR mit demselben Wert hebt sich auf.
- S^{-1} ist die S-Box „von rechts nach links" gelesen, P^{-1} die inverse Umsortierung.

Im Mini-Beispiel (Runde 2 rückwärts):

Schritt	Rechnung	Ergebnis
P^{-1}	(y_2, y_4, y_1, y_3) auf 1101	1110
S^{-1}	$[11][10] \to [01][11]$	0111
$\oplus k_2$	$0111 \oplus 1001$	1110

Dann Runde 1 rückwärts analog → man landet exakt wieder bei $x = 1011$. ✓

> **Darum muss die S-Box bijektiv sein**
>
> Wäre sie es nicht, gäbe es für manche Eingaben mehrere Urbilder → man könnte nicht eindeutig entschlüsseln.

7. Warum mehrere Runden so stark sind

Intuition: **Teig kneten.** Einmal falten und durchkneten vermischt noch nicht alles; nach vielen Runden ist jedes Krümelchen Mehl mit allem verbunden. Genau das passiert mit den Bits: nach genug Runden hängt **jedes** Ausgabebit von **vielen** Eingabebits und allen Subkeys ab. Ein einzelnes gekipptes Eingabebit verändert dann praktisch den ganzen Ciphertext (Lawineneffekt).

8. In der echten Welt

- **AES** ist im Kern ein erweiterter SPN: 128-Bit-Block, 8-Bit-S-Box, 10–14 Runden.
- **DES** ist dagegen ein **Feistel-Netzwerk** (siehe FeistelNetwork_Explanation) — anderer Aufbau, aber dieselbe Grundphilosophie: Konfusion + Diffusion durch Runden.

9. Zusammenfassung

✓ **Die drei Kernaussagen**

1. **Zwei Bausteine:** Substitution (nichtlinear → Konfusion) und Permutation (linear → Diffusion).
2. **Viele Runden** verketten beides, bis der Output zufällig aussieht.
3. **Nur die Key-Addition** trägt den Schlüssel; S und P sind öffentlich. Darauf beruht der Angriff in Q3.

Hier geht's zurück zur normalen Zusammenfassung!

FeistelNetwork_Explanation

1. Das Grundproblem, das Feistel löst

Eine **Blockchiffre** ist im Kern:

- nimm einen Block fester Länge (z.B. 64 Bit),
- mit einem Schlüssel k,
- verwandle ihn **umkehrbar** (bijektiv) in einen Ciphertext.

„Umkehrbar" ist der schwierige Teil. Stell dir vor, du willst einen starken Cipher aus einfachen Bausteinen bauen. Wenn du einfach beliebige Funktionen hintereinander hängst, ist das Ergebnis **nicht mehr umkehrbar** — es sei denn, **jede** einzelne Komponente ist invertierbar. Genau das macht eine SPN (AES): jede S-Box, jede Permutation **muss** invertierbar sein.

> **Feistels geniale Idee**
>
> Kann man eine **umkehrbare** Chiffre bauen, deren Baustein f **überhaupt nicht** umkehrbar sein muss?

2. Der Aufbau

Man nimmt den Block und **spaltet ihn in zwei Hälften**:

- Blocklänge $2n$ Bit → linke Hälfte L, rechte Hälfte R, jeweils n Bit.
- Bei DES: $2n = 64$, also $n = 32$ (je 32-Bit-Hälften) → 📖 IntroCrypto3, S. 9.

Und dann immer dieselbe Runde, r mal wiederholt.

3. Eine einzelne Runde — Schritt für Schritt

Die Rundenregeln (Folie: 📖 IntroCrypto3, S. 10):

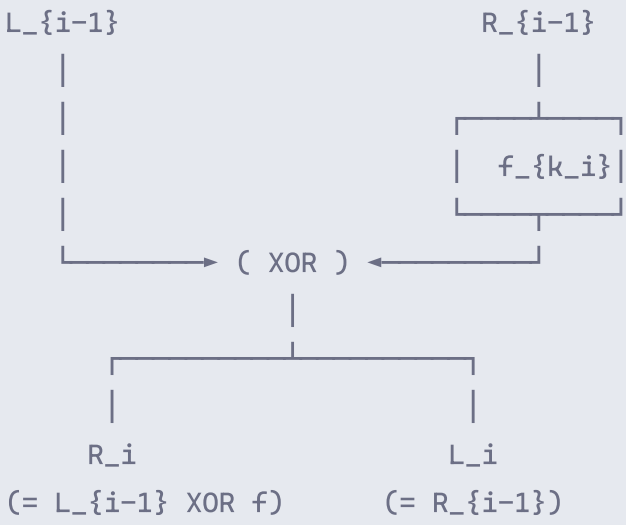
$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f_{k_i}(R_{i-1})$$

Was da **wirklich** passiert, in Worten:

1. **Die neue linke Hälfte ist einfach die alte rechte Hälfte.** $L_i = R_{i-1}$. Nichts rechnen, nur rüberschieben.
2. **Aus der alten rechten Hälfte wird ein „Mischwert" gebaut:** $f_{k_i}(R_{i-1})$. Das ist die Rundenfunktion f mit dem Rundenschlüssel k_i.

3. **Dieser Mischwert wird per XOR in die alte linke Hälfte eingerührt:** $R_i = L_{i-1} \oplus f_{k_i}(R_{i-1})$.

Als Bild (die beiden Hälften „überkreuzen" sich):



Also: **eine Hälfte geht unverändert weiter, die andere wird verwürfelt.** Deshalb „crossing"/swap.

> **Warum XOR?**
>
> XOR ist seine **eigene Umkehrung**: $a \oplus b \oplus b = a$. Wenn ich b in a eingerührt habe, kann ich es mit **demselben** b wieder herausnehmen. Genau darauf baut die Entschlüsselung.

4. Der Trick: Entschlüsseln ohne f umkehren zu können

Angenommen ich kenne den Output (L_i, R_i) einer Runde und will den Input (L_{i-1}, R_{i-1}) zurück.

1. Aus Regel 1 wissen wir: $L_i = R_{i-1}$. Also ist die **alte rechte Hälfte sofort bekannt**: $R_{i-1} = L_i$. (Wir müssen nichts „rückwärts" bewerten!)
2. Jetzt kenne ich R_{i-1}, also kann ich $f_{k_i}(R_{i-1})$ **normal vorwärts** ausrechnen. Und dann:

$$L_{i-1} \oplus f_{k_i}(R_{i-1}) = R_i \quad \Longrightarrow \quad L_{i-1} = R_i \oplus f_{k_i}(R_{i-1})$$

Da $R_{i-1} = L_i$ gilt, schreibt man das oft als:

$$L_{i-1} = R_i \oplus f_{k_i}(L_i), \qquad R_{i-1} = L_i$$

> **Der entscheidende Punkt**
>
> Wir brauchen f **nie rückwärts**. Wir rufen f_{k_i} immer **vorwärts** auf einen Input auf, den wir **schon kennen** (L_i), und heben dann mit XOR auf. Deshalb darf f total chaotisch und **nicht invertierbar** sein — es stört niemanden.

5. Konkretes Mini-Beispiel (klein genug zum Nachrechnen)

Nehmen wir $n = 4$ Bit pro Hälfte (Block = 8 Bit) und eine Spielzeug-Rundenfunktion

$$f_k(R) = (R + k) \bmod 16.$$

Eingabe: $L_0 = 3 = 0011$, $R_0 = 5 = 0101$, erster Rundenschlüssel $k_1 = 2$.

Verschlüsseln (1 Runde):

Schritt	Rechnung	Ergebnis
$L_1 = R_0$	–	5
$f_{k_1}(R_0)$	$(5 + 2) \bmod 16$	7
$R_1 = L_0 \oplus f_{k_1}(R_0)$	$3 \oplus 7$	4

Output: $(L_1, R_1) = (5, 4)$.

Entschlüsseln (dieselbe f, dieselbe k_1):

Schritt	Rechnung	Ergebnis
$R_0 = L_1$	–	5
$f_{k_1}(L_1)$	$(5 + 2) \bmod 16$	7
$L_0 = R_1 \oplus f_{k_1}(L_1)$	$4 \oplus 7$	3

Wir sind exakt wieder bei $(L_0, R_0) = (3, 5)$. ✓ — Beachte: Wir haben f **genau gleich** wie beim Verschlüsseln benutzt, nichts invertiert.

6. Viele Runden & Rundenschlüssel

- Man wiederholt die Runde r mal.
- Jede Runde benutzt ihren **eigenen** Rundenschlüssel k_i, abgeleitet aus dem Master-Key (Key Schedule). Bei DES: $r = 16$ → 📖 IntroCrypto3, S. 11.
- **Entschlüsseln** = genau dasselbe Netzwerk, nur die Rundenschlüssel in **umgekehrter Reihenfolge** $k_r, k_{r-1}, \ldots, k_1$.

☰ Warum überhaupt mehrere Runden?

Eine Runde „vermischt" nur eine 32-Bit-Hälfte. Erst durch viele Runden zieht sich die Verwirrung über den ganzen Block. Und **wie viele** nötig sind, ist genau der Punkt von Q2 (b)/(c):

- **1 Runde:** offensichtliches Leck — die linke Ausgabe-Hälfte ist einfach die rechte Eingabe-Hälfte ($L_1 = R_0$).
- **2 Runden:** immer noch ein Leck (zwei Eingaben mit gleicher rechter Hälfte verraten eine XOR-Relation).
- **3 Runden:** erst jetzt „sieht es aus wie zufällig" (Luby–Rackoff, PRP).
- **4 Runden:** sogar strong PRP.
  Deshalb hat DES **16** Runden — Sicherheitsmarge.

7. Warum das genial ist (Zusammenfassung)

✓ Die drei Kernaussagen

1. **Man braucht kein invertierbares f.** Nur f selbst (vorwärts), nie f^{-1}.
2. **Invertierbarkeit ist „gratis".** Sie entsteht allein dadurch, dass $L_i = R_{i-1}$ und das XOR-Einrühren mit demselben Wert aufgehoben wird.
3. **Stärke kommt von den Runden.** Wenige Runden leaken (Q2 b/c), viele Runden sind sicher.

Hier geht's zurück zur normalen Zusammenfassung!